SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

INSTITUTE OF ECE

ECA14 – EMBEDDED SYSTEMS

List of Experiments

Exp. No	Name of the Experiment	Remarks
1	Flashing of Led using AT89C51 Microcontroller using Proteus	
2	Generation of Square Wave using Proteus	
3	Stepper Motor Using AT89C51 using Proteus	
4	7 Segment Display Using AT89C51 using Proteus	
5	Arduino based Manual Electronic Counter using Proteus	
6	Arduino to 16x2 LCD Display using Proteus	
7	Serial Communication using Arduino with Proteus	
8	Gas Sensor MQ-2 in Proteus using Arduino	
9	Ultrasonic Sensor in Proteus using Arduino	
10	PIR Sensor in Proteus using Arduino	
11	Study of GPIO & Bit Addressing in AT89C51 using Embedded C	
12	Embedded C program to multiply two 16-bit binary numbers.	
13	Embedded C program to add an array of 16-bit numbers and store the 32-bit result in internal RAM.	
14	Embedded C program to display “Hello World” message	
15	Embedded C program to convert the hexadecimal data to decimal and display the digits on ports P0, P1 and P2	
16	Serial Interrupt Using UART (Keyboard Input) In Keil	
17	UART Interfacing using Interrupt in KEIL	

18	Generation of PWM using TIMER Interrupt	
19	Blinking LEDs using software delay routine in LPC2148 kit	

20	Program to read the switch and display in the LEDs using LPC2148 kit	
21	Display a number in seven segment LED in LPC2148 kit	
22	Serial transmission and reception using on-chip UART in LPC2148 kit	
23	Access an internal ADC and display the binary output in LEDs in LPC2148 kit	
24	Modeling Clock-Driven Concurrent Processes Using SC_THREAD and wait() in SystemC	
25	Producer–Consumer Problem Using SystemC	
26	Transaction Level Modeling (TLM) using SystemC	
27	Binary Semaphore in SystemC	
28	Counting Semaphore in SystemC	
29	Priority-Based Interrupt Handling	
30	RTOS Task Scheduling	

STUDY OF PROTEUS AND KEIL MICRO VISION

AIM: To study the working procedures of Proteus and Keil Micro vision software.

Keil Micro Vision is a free software which solves many of the main points for an embedded program developer. This software is an integrated development environment (IDE), which integrated a text editor to write programs, a compiler and it will convert your source code to hex files too. µVision4 introduces a flexible window management system, enabling us to drag and drop individual windows anywhere on the visual surface including support for Multiple Monitors.

KEIL PROCEDURE:

1. Open the software, Click on project and open new version project.
2. Create a new project file
3. Enter AT89C51
4. Click NO
5. Click [Ctrl +N] and Type the code
6. Open project and click Build target
7. Open Build target and open source file and ADD, CLOSE
8. Click build target
9. Next debug start and stop
10. Open peripherals and select port 2
11. Now run the program in Debug
12. Open project and click optional properties and in that give output as hex file.
13. Create hex file.

PROTEUS PROCEDURE:

- Open proteus by clicking run as administrator.
- Open new project and enter the file name.
- Click next, next, next and finish.
- Click P symbol and search keyword and place the required components • Now connect the components as required
- Give input to AT89C51 as HEX file.
- Start the simulation process

RESULT: Thus the Proteus and Keil Micro vision software were studied.

Exp No: 1

FLASHING OF LED USING AT89C51 MICROCONTROLLER USING PROTEUS

AIM:

To Write an assembly language program to flash the LED using AT89C51

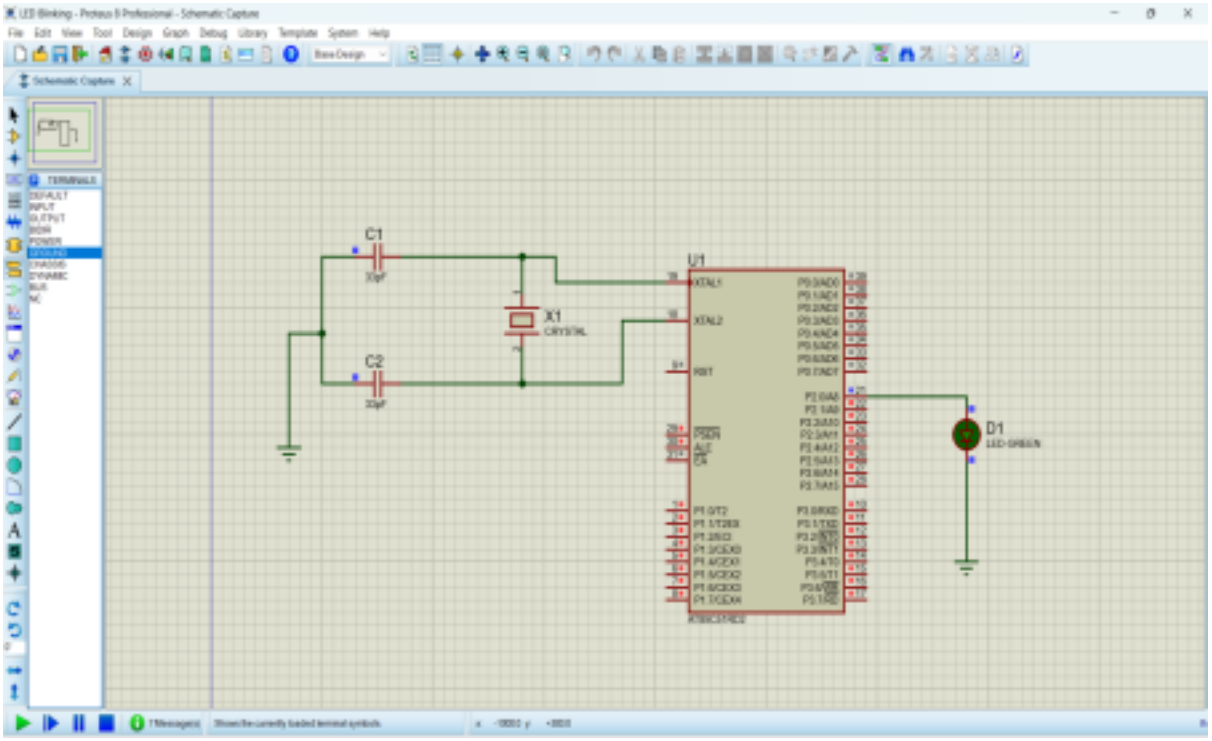
SOFTWARES REQUIRED:

- Proteus software

PROGRAM

```
        ORG 0000H
UP: SETB P2.0
        ACALL DELAY
        CLR P2.0
        ACALL DELAY
        SJMP UP
DELAY: MOV R4,#35
H1:MOV R3,#255
H2:DJNZ R3,H2
        DJNZ R4,H1
        RET
        END
```

CIRCUIT DIAGRAM:



RESULT

Thus the program has been successfully verified and executed.

Exp No: 2

GENERATION OF SQUARE WAVE USING PROTEUS

AIM:

To write an assembly language program to Generate square wave using AT89C51.

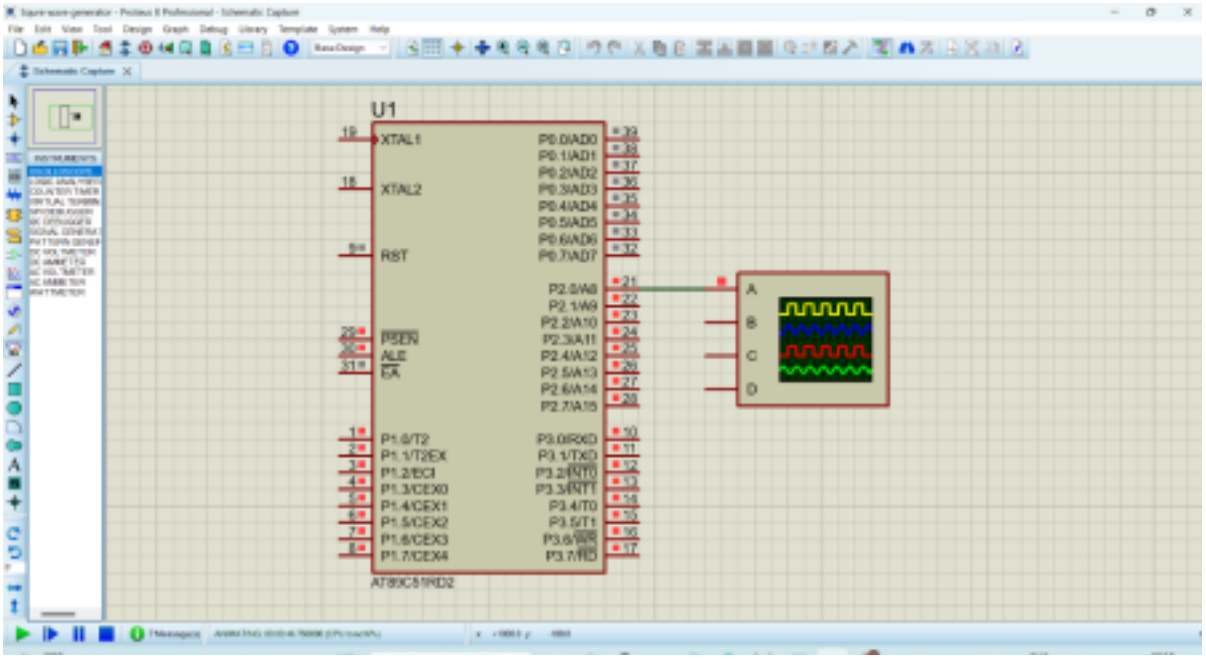
SOFTWARE REQUIRED:

- Proteus 8 software.

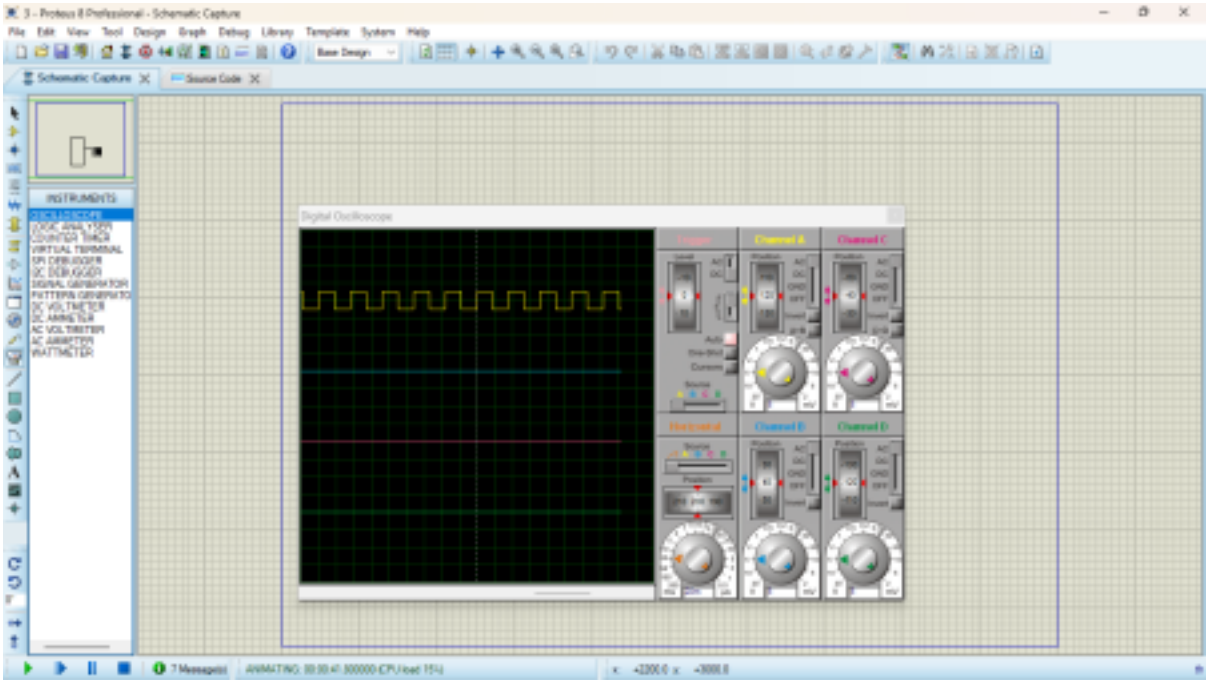
PROGRAM

```
        ORG 0000H
UP: SETB P2.0
        ACALL DELAY
        CLR P2.0
        ACALL DELAY
        SJMP UP
DELAY: MOV R4,#35
H1:MOV R3,#255
H2:DJNZ R3,H2
        DJNZ R4,H1
        RET
        END
```

CIRCUIT DIAGRAM:



OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 3

STEPPER MOTOR USING AT89C51 USING PROTEUS

AIM:

To write an assembly language program for Stepper Motor Using AT89C51 using Keil and Proteus

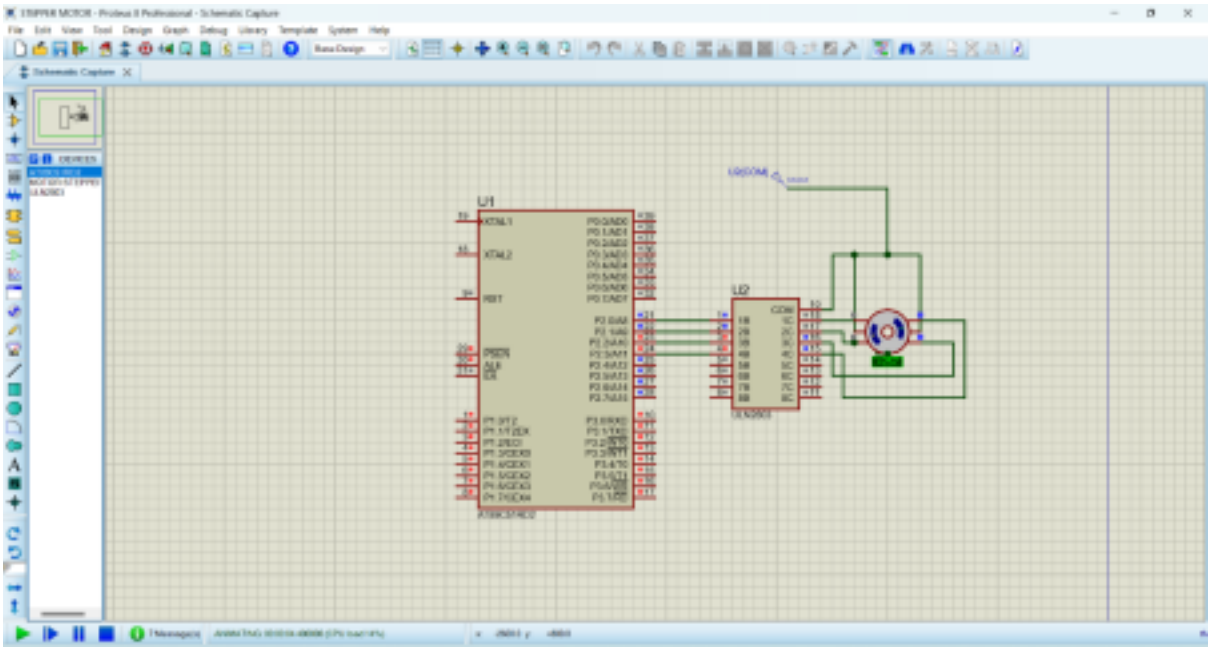
SOFTWARE REQUIRED:

- Proteus 8 software.

PROGRAM:

```
ORG 0000H
UP: MOV P2,#09H
ACALL DELAY
MOV P2,#0CH
ACALL DELAY
MOV P2,#06H
ACALL DELAY
MOV P2,#03H
ACALL DELAY
SJMP UP
DELAY:MOV R4,#18
H1:MOV R3,#255
H2:DJNZ R3,H2
DJNZ R4,H1
RET
END
```

CIRCUIT DIAGRAM:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 4

7 SEGMENT DISPLAY USING AT89C51 USING PROTEUS

AIM:

To write an assembly language program for 7 Segment Display Using AT89C51 using Keil and Proteus

SOFTWARE REQUIRED:

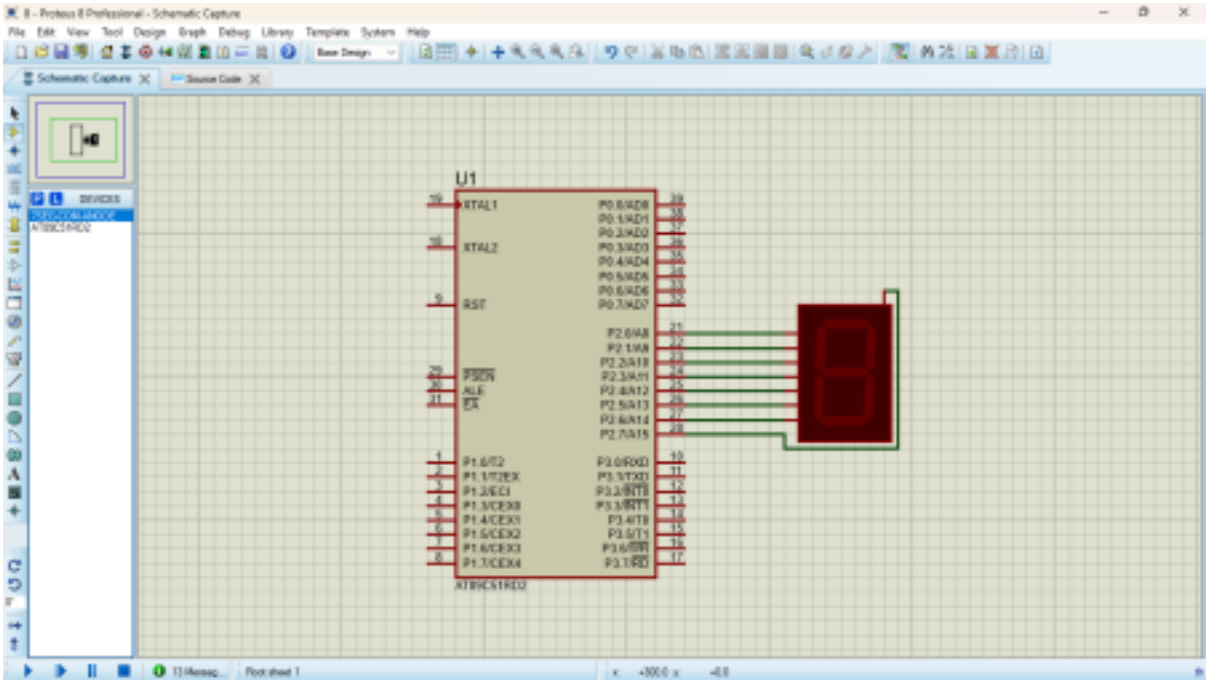
- Proteus 8 software.

PROGRAM:

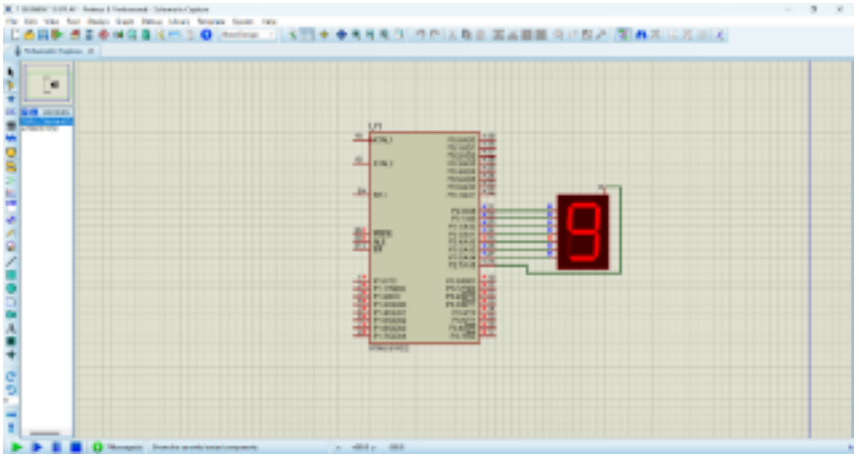
```
ORG 000H
UP:MOV P2,#0C0H
ACALL DELAY
MOV P2,#0F9H
ACALL DELAY
MOV P2,#0A4H
ACALL DELAY
MOV P2,#0B0H
ACALL DELAY
MOV P2,#99H
ACALL DELAY
MOV P2,#92H
ACALL DELAY
MOV P2,#82H
ACALL DELAY
MOV P2,#0F8H
ACALL DELAY
MOV P2, #80H
ACALL DELAY
MOV P2,#90H
ACALL DELAY

DELAY: MOV R5,#10
H1:MOV R4,#180
H2:MOV R3,#255
H3:DJNZ R3,H3
DJNZ R4,H2
DJNZ R5,H1
RET
END
```

CIRCUIT DIAGRAM:



OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 9

ARDUINO BASED MANUAL ELECTRONIC COUNTER USING

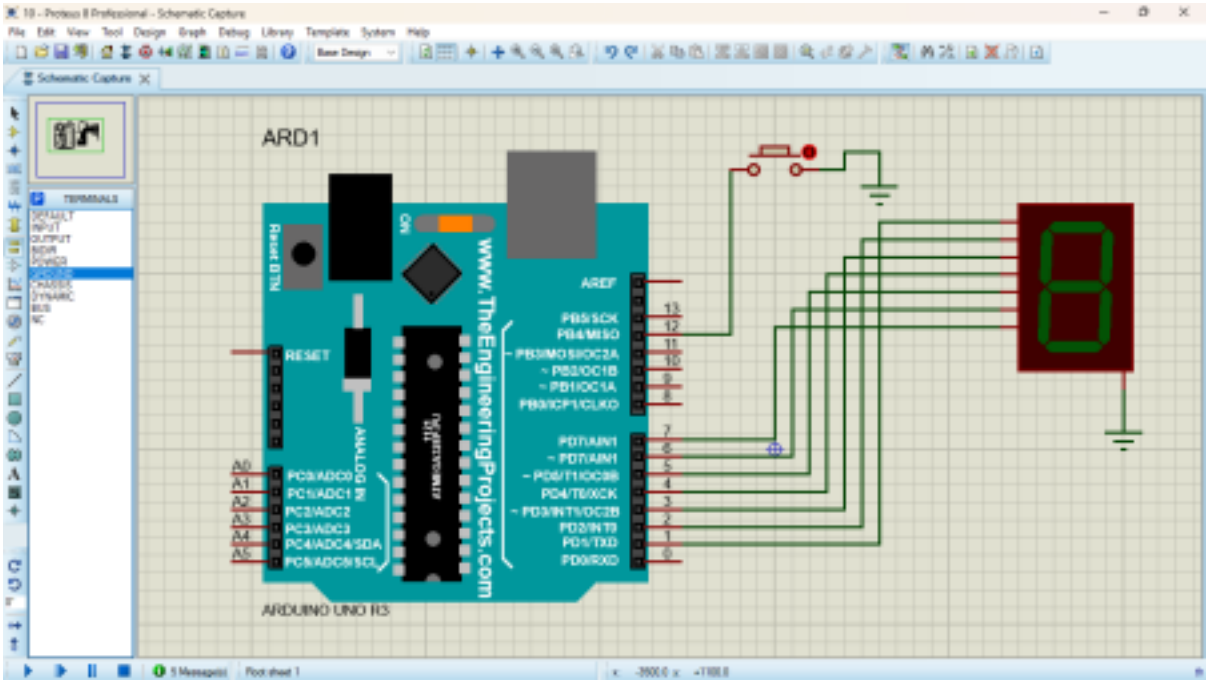
PROTEUS AIM:

To write an Embedded C program for Manual Electronic counter using Arduino IDE and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software
- Arduino IDE

CIRCUIT DIAGRAM:



PROGRAM:

```
int x0,x1,x2,x3,x4,x5,x6,x7,x8,x9;

int delay_time=200;


void setup() {

 //configure pin2 as an input and enable the internal pull-up resistor

 pinMode(12, INPUT_PULLUP);

 pinMode(1,OUTPUT);
 pinMode(2,OUTPUT);

 pinMode(3,OUTPUT);

 pinMode(4,OUTPUT);

 pinMode(5,OUTPUT);

 pinMode(6,OUTPUT);

 pinMode(7,OUTPUT);


}

void loop() {
```

```
 //read the pushbutton value into a variable
int sensorVal = digitalRead(12);


 if (sensorVal == LOW) {
 x0=true;
 }
 while(x0){
 zero();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {
 x1=true;
 x0=false;
 }
 }
 while(x1){
 one();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {
 x2=true;
 x1=false;
 }
 }
 while(x2){
 two();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x3=true;
 x2=false;
 }
 }
  while(x3){
```

```
 three();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x4=true;
 x3=false;
 }
 }
 while(x4){
 four();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x5=true;
 x4=false;
 }
 }
 while(x5){
 five();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x6=true;
 x5=false;
 }
 }
 while(x6){
 six();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x7=true;
 x6=false;
 }
 }
 while(x7){
 seven();
```

```
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x8=true;
 x7=false;
 }
 }
 while(x8){
 eight();
 sensorVal = digitalRead(12);  if
(sensorVal == LOW) {  x9=true;
 x8=false;
 }
 }
 while(x9){
 nine();
 sensorVal = digitalRead(12);
 if (sensorVal == LOW) {
x0=true;
 x9=false;
 }
 }


 }


 void zero()
 {
 for(int i=1;i<7;i++)
 {
   digitalWrite(i,HIGH);
digitalWrite(7,LOW);
```

```
}
 delay(delay_time);
 }

 void one()
 {
    digitalWrite(1,LOW);
digitalWrite(2,HIGH);
digitalWrite(3,HIGH);
digitalWrite(4,LOW);
digitalWrite(5,LOW);
digitalWrite(6,LOW);
digitalWrite(7,LOW);
delay(delay_time);
 }
 void two()
 {
   digitalWrite(1,HIGH);
digitalWrite(2,HIGH);
digitalWrite(3,LOW);
digitalWrite(4,HIGH);
digitalWrite(5,HIGH);
digitalWrite(6,LOW);
 digitalWrite(7,HIGH);
delay(delay_time);  }

 void three()
 {
   digitalWrite(1,HIGH);
digitalWrite(2,HIGH);
```

```
digitalWrite(3,HIGH);
digitalWrite(4,HIGH);
digitalWrite(5,LOW);
digitalWrite(6,LOW);
 digitalWrite(7,HIGH);
delay(delay_time);  }

 void four()
 {
 digitalWrite(1,LOW);
digitalWrite(2,HIGH);
digitalWrite(3,HIGH);
digitalWrite(4,LOW);
 digitalWrite(5,LOW);
digitalWrite(6,HIGH);
digitalWrite(7,HIGH);
 delay(delay_time);
}

 void five()
 {
    digitalWrite(1,HIGH);
digitalWrite(2,LOW);
digitalWrite(3,HIGH);
digitalWrite(4,HIGH);
digitalWrite(5,LOW);
 digitalWrite(6,HIGH);
digitalWrite(7,HIGH);
delay(delay_time);  }
```

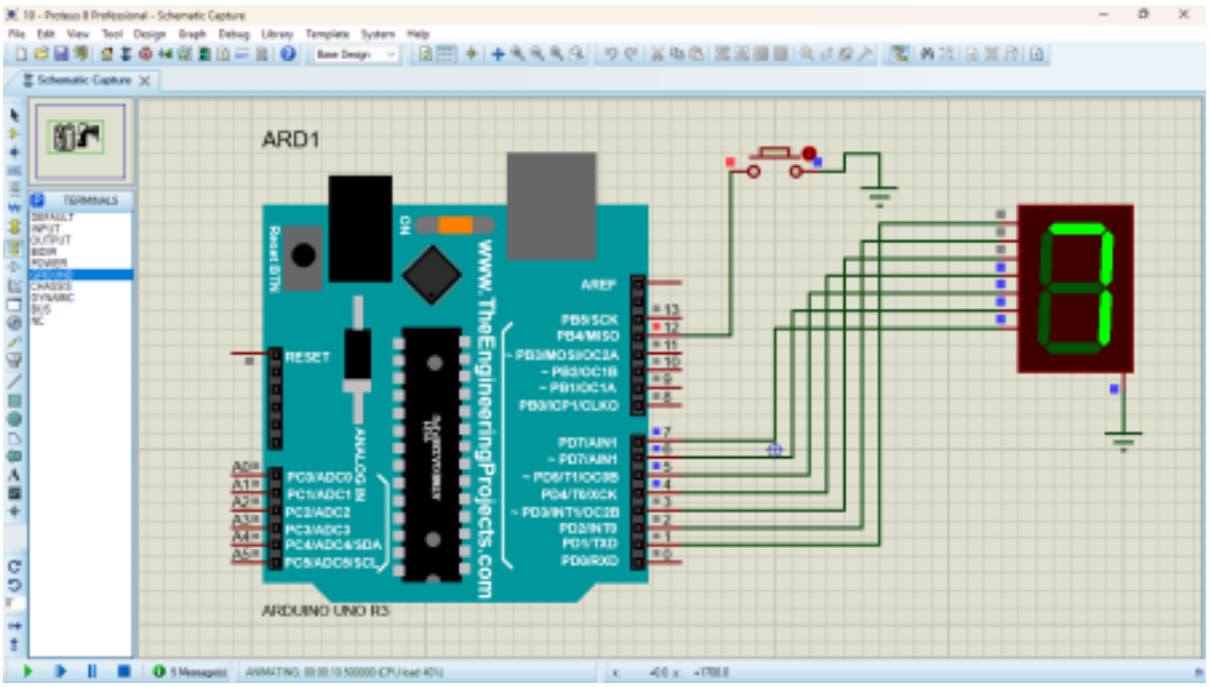
```
 void six()
 {
   digitalWrite(1,HIGH);
digitalWrite(2,LOW);
digitalWrite(3,HIGH);
digitalWrite(4,HIGH);
digitalWrite(5,HIGH);
digitalWrite(6,HIGH);
digitalWrite(7,HIGH);
delay(delay_time);
 }
 void seven()
 {
   digitalWrite(1,HIGH);
digitalWrite(2,HIGH);
digitalWrite(3,HIGH);
digitalWrite(4,LOW);
digitalWrite(5,LOW);
 digitalWrite(6,LOW);
digitalWrite(7,LOW);
delay(delay_time);  }

 void eight()
 {
   digitalWrite(1,HIGH);
digitalWrite(2,HIGH);
digitalWrite(3,HIGH);
digitalWrite(4,HIGH);
digitalWrite(5,HIGH);
```

```
digitalWrite(6,HIGH);
digitalWrite(7,HIGH);
delay(delay_time);
 }

 void nine()
 {
    digitalWrite(1,HIGH);
digitalWrite(2,HIGH);
digitalWrite(3,HIGH);
digitalWrite(4,HIGH);
 digitalWrite(5,LOW);
 digitalWrite(6,HIGH);
 digitalWrite(7,HIGH);
 delay(delay_time);
 }
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 6

ARDUINO TO 16x2 LCD DISPLAY USING PROTEUS

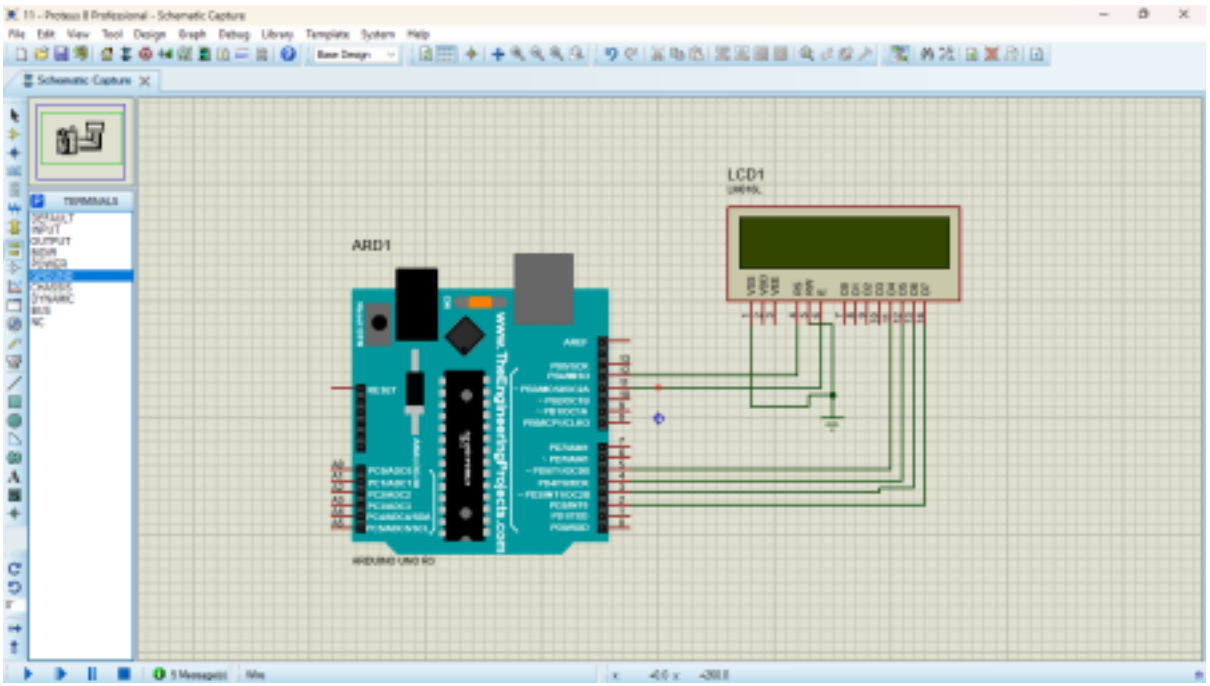
AIM:

To write an Embedded C program to interface Arduino Uno and 16 x 2 LCD display using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:



PROGRAM:

Arduino to 16x2 LCD Display using Proteus

```
#include <LiquidCrystal.h>
LiquidCrystal lcd (7, 8, 9, 10, 11, 12);
// your PWM numbers may be different

void setup() {

lcd.begin(16, 2);
```

```
lcd.print("put your message here");

}

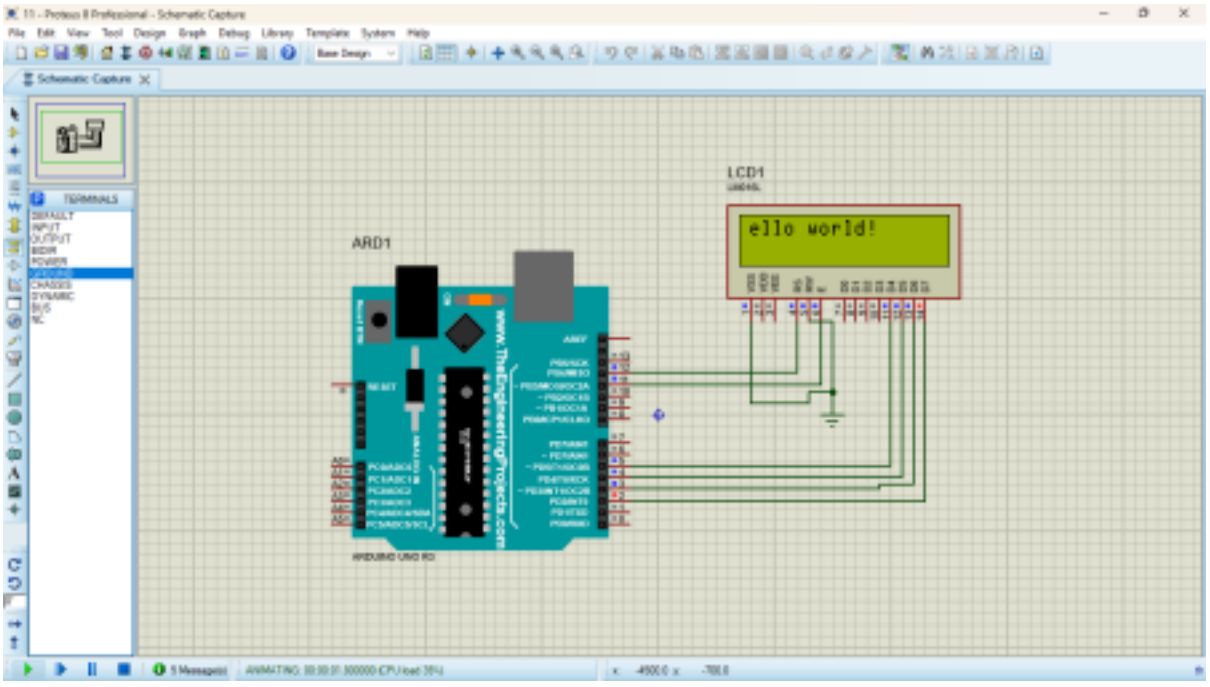
void loop () {

for (int i=0; 1<13; i++)
}

lcd.scrollDisplayLeft();
delay(700);

}
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 7

SERIAL COMMUNICATION USING ARDUINO WITH PROTEUS

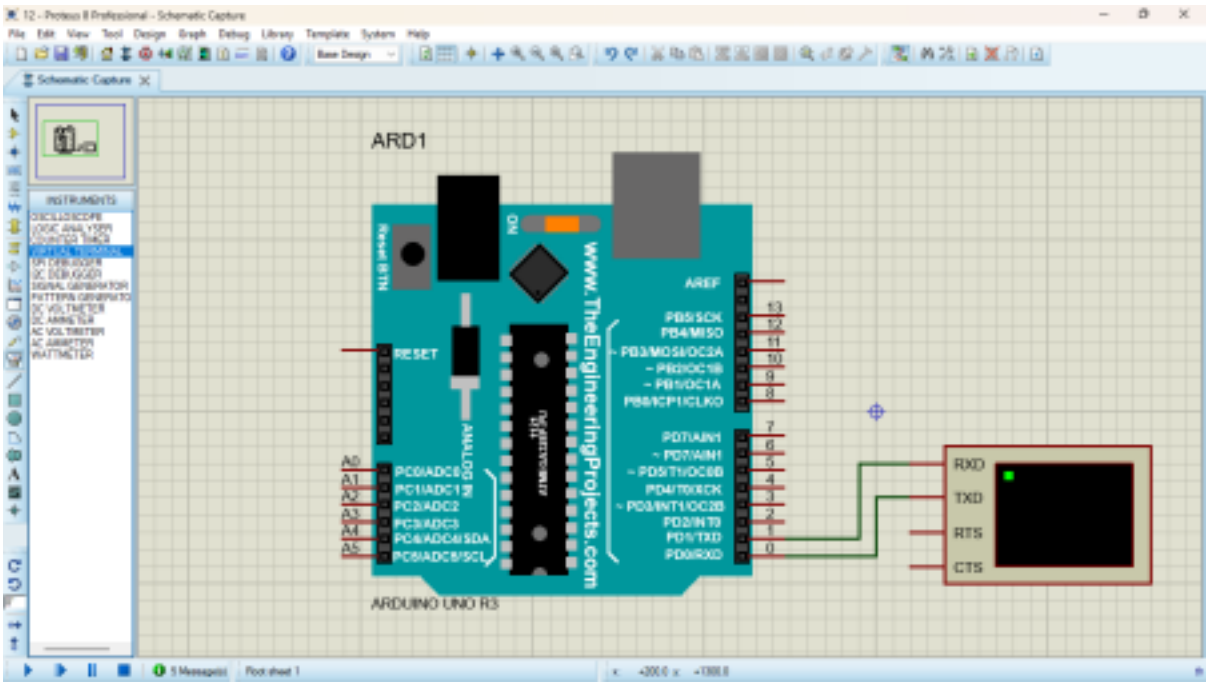
AIM:

To write an Embedded C program for serial communication using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:



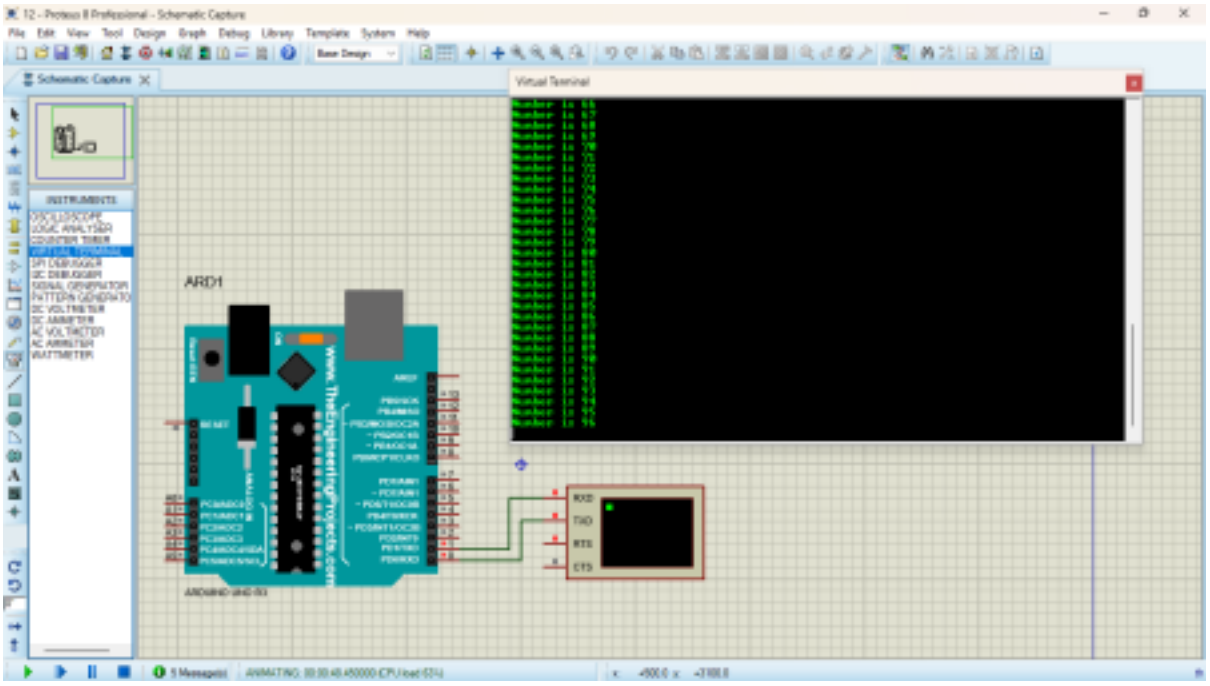
PROGRAM:

```
void setup()
 {
Serial.begin(9600); // send and receive at 9600 baud
}


int number = 0;
void loop()
 {
Serial.print("Number is ");
Serial.println(number); // print the number
```

```
delay(500); // delay half second between numbers
number++; // to the next number
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 8

GAS SENSOR MQ-2 IN PROTEUS USING ARDUINO

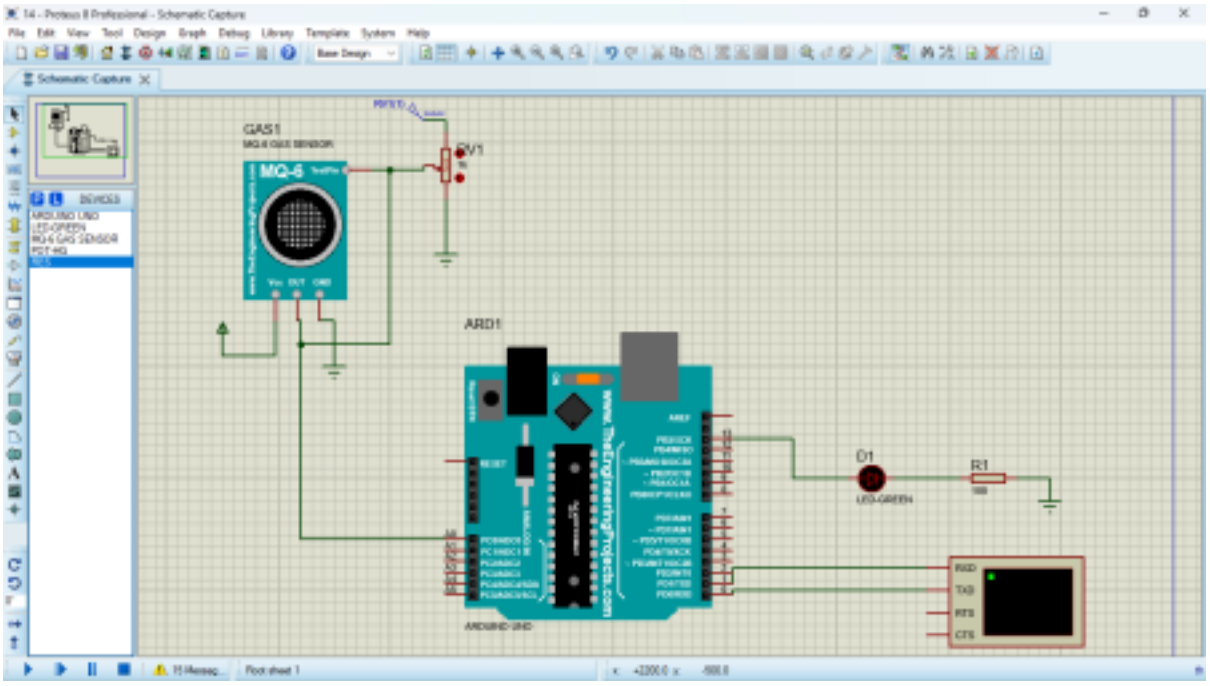
AIM:

To write an Embedded C program for interfacing Gas Sensor MQ-2 using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:



PROGRAM:

```
#define GreenLed 13
#define Sensor A0


void setup(){
 pinMode(13,OUTPUT);
 pinMode(12,OUTPUT);
 pinMode(A0,INPUT);
 Serial.begin(9600);
}
void loop() {
 int value = analogRead(A0);
 Serial.print("Analogic Value coming from the sensor : ");
Serial.println(value);
 delay(100);
 if(value>600){
 digitalWrite(13,HIGH);
 }
 else{
```

```
 digitalWrite(13,LOW);
 }
 delay(20);
}
```

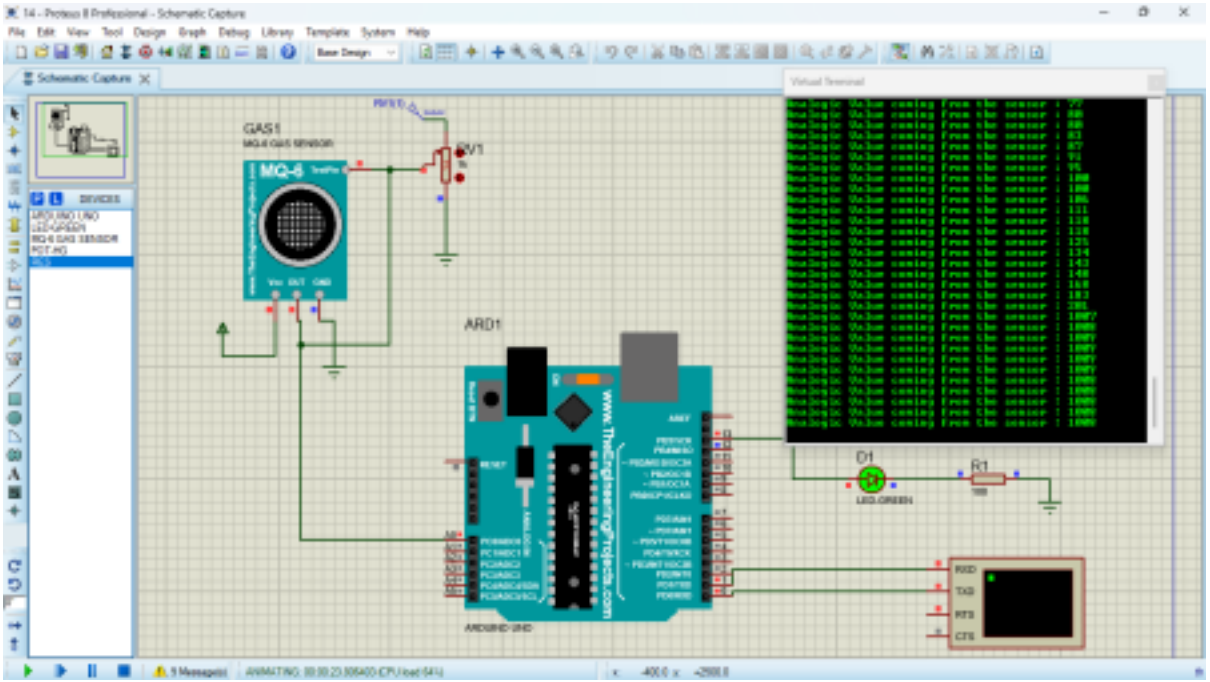
Code for MQ6 sensor:

```
int mq6Pin = A0; // MQ-5 analog pin connected to A0

void setup() {
 Serial.begin(9600);
}

void loop() {
 int sensorValue = analogRead(mq5Pin);
 Serial.print("MQ5 Sensor Value: ");
 Serial.println(sensorValue);
 delay(1000); // 1-second delay
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 9

ULTRASONIC SENSOR IN PROTEUS USING ARDUINO

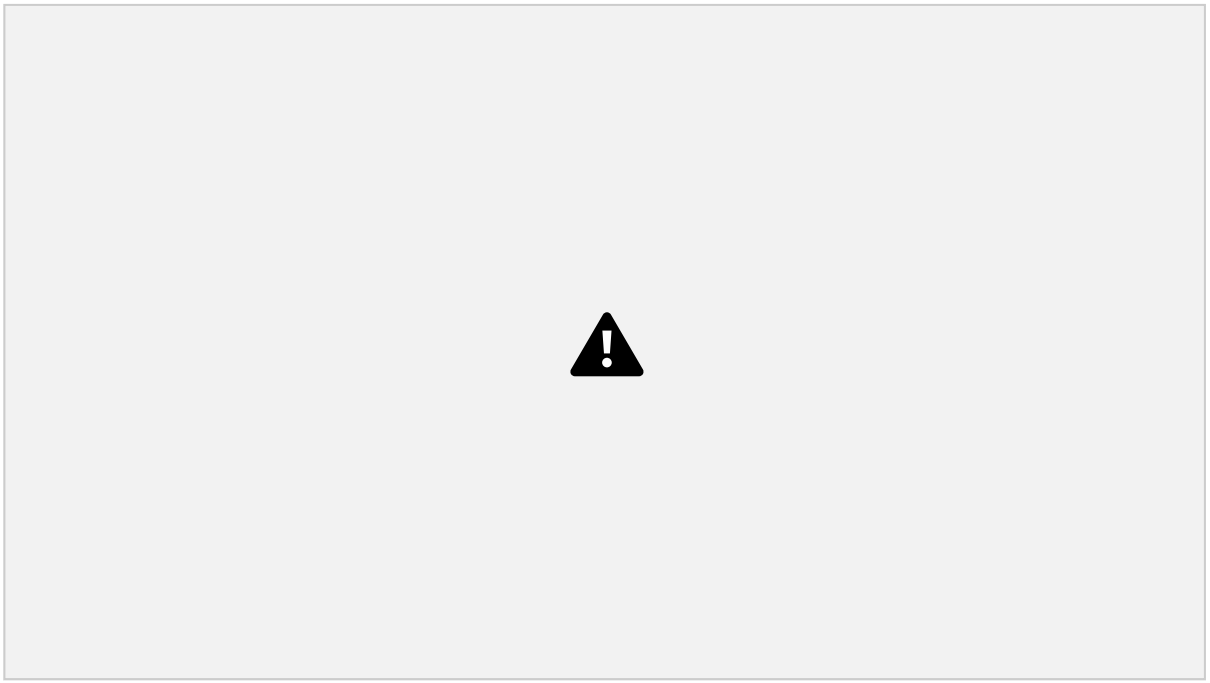
AIM:

To write an Embedded C program for interfacing Ultrasonic Sensor using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:

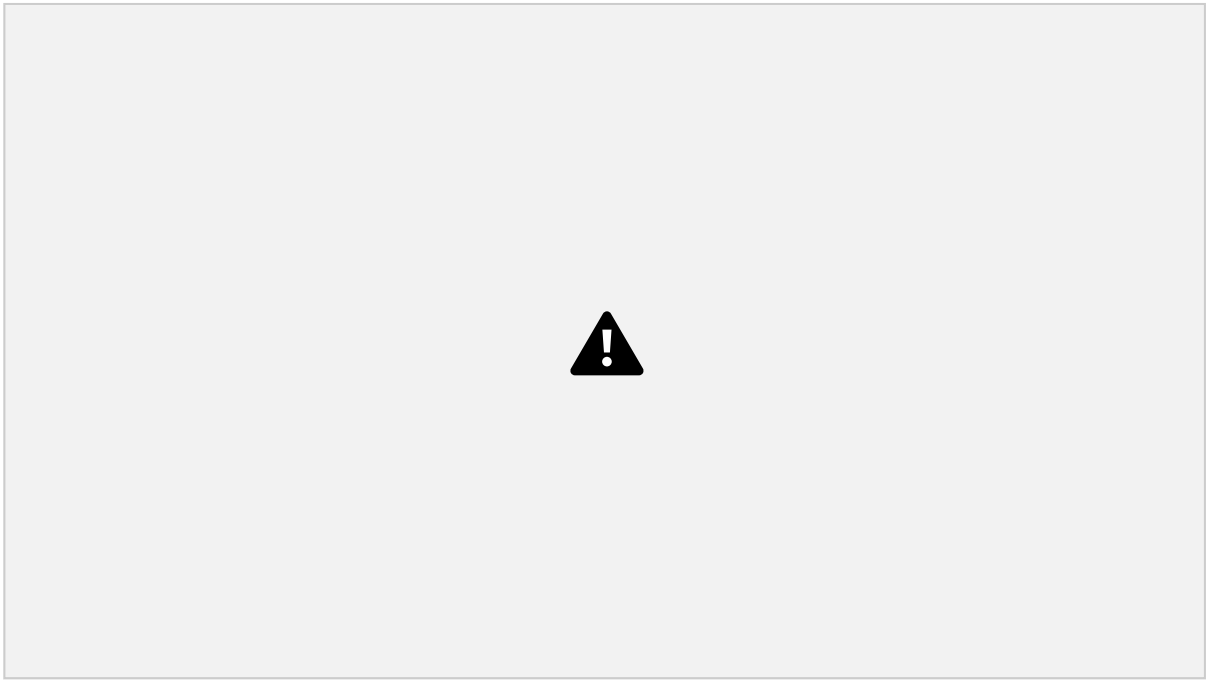


PROGRAM:

```
// Program Sketch
 int trig = 10;
 int echo = 9;
 long duration;
 int cm;
void setup() {
 // put your setup code here, to run once:
 pinMode(trig, OUTPUT);
```

```
 pinMode(echo,INPUT);
 Serial.begin(9600);
 }
void loop() { // Sketch to calculate ultrasonic distance
 digitalWrite(trig, LOW);
 delayMicroseconds(10);
 digitalWrite(trig, HIGH);
 delayMicroseconds(10);
 digitalWrite(trig, LOW);
 delayMicroseconds(10);
 duration = pulseIn(echo, HIGH);
 cm = (duration/2) * 0.034;
 // Print Value on Serial Monitor
 Serial.print(" Distance = ");
 Serial.print(cm);
 Serial.println(" cm");
 }
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 10

PIR SENSOR IN PROTEUS USING ARDUINO

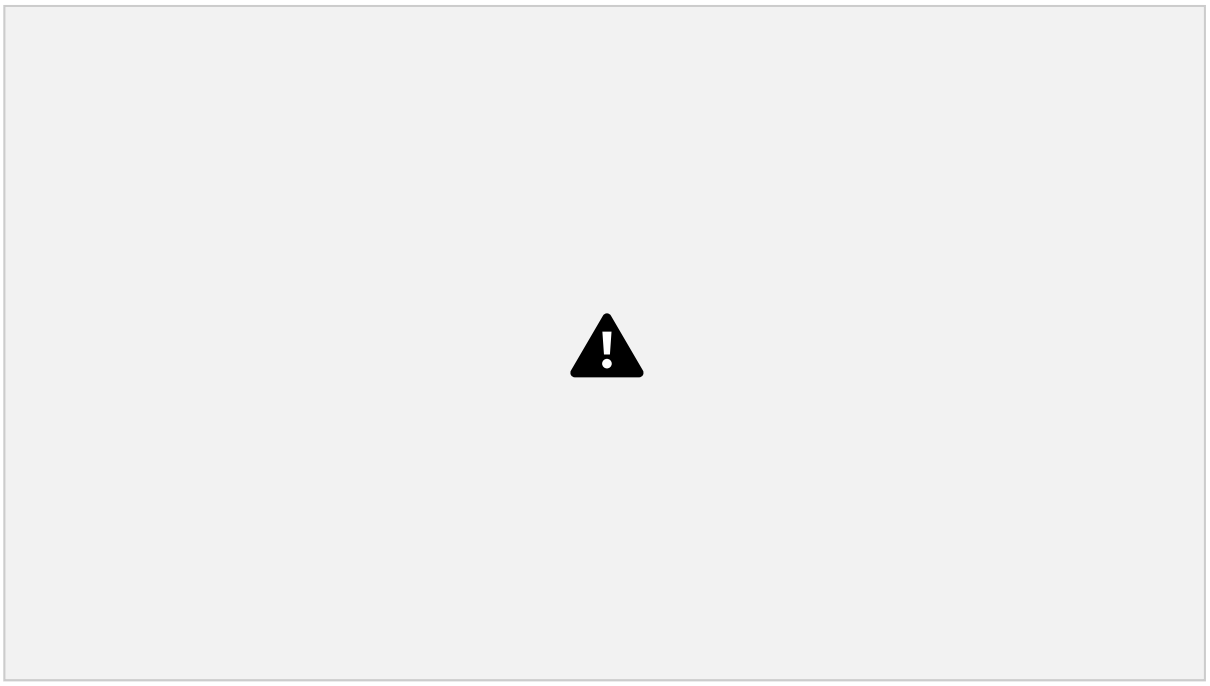
AIM:

To write an Embedded C program for interfacing PIR Sensor using Arduino Uno and Proteus

SOFTWARE REQUIRED:

- Proteus 8 software.
- Arduino IDE software

CIRCUIT DIAGRAM:

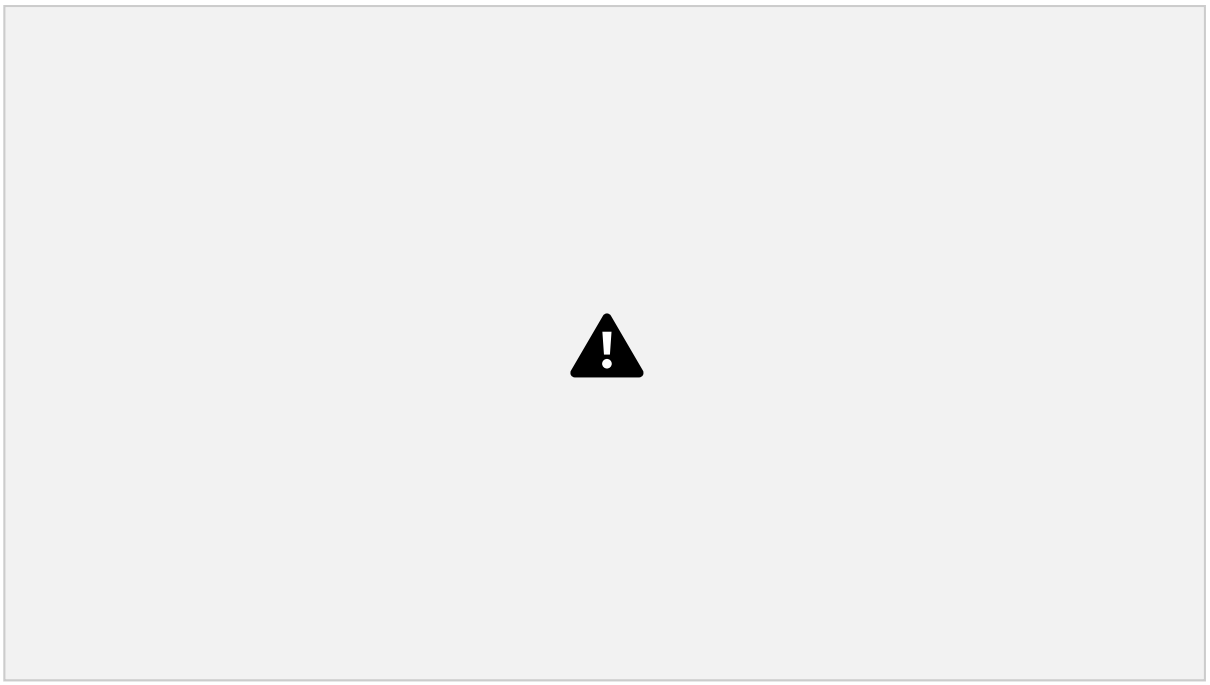


PROGRAM:

```
int Status = 13; // Digital pin D6
int sensor = 12; // Digital pin D7
void setup() {
 Serial.begin(9600);
 pinMode(sensor, INPUT); // declare sensor as input
 pinMode(Status, OUTPUT); // declare LED as output
}
```

```
void loop() {
 long state = digitalRead(sensor);
 Serial.println(state);
 if(state == HIGH) {
 digitalWrite (Status, HIGH);
 Serial.println("Motion detected!");
 delay(1000);
 }
 else {
 digitalWrite (Status, LOW);
 Serial.println("Motion absent!");
 delay(1000);
 }
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 11

STUDY OF GPIO & BIT ADDRESSING IN AT89C51

AIM:

To study GPIO control and bit-addressable SFR using Embedded C.

SOFTWARE REQUIRED:

- Keil

THEORY

8051 ports are byte and bit addressable. Using sbit, individual bits of a port can be accessed directly. Ports default to logic HIGH due to internal pull-ups.

PROGRAM:

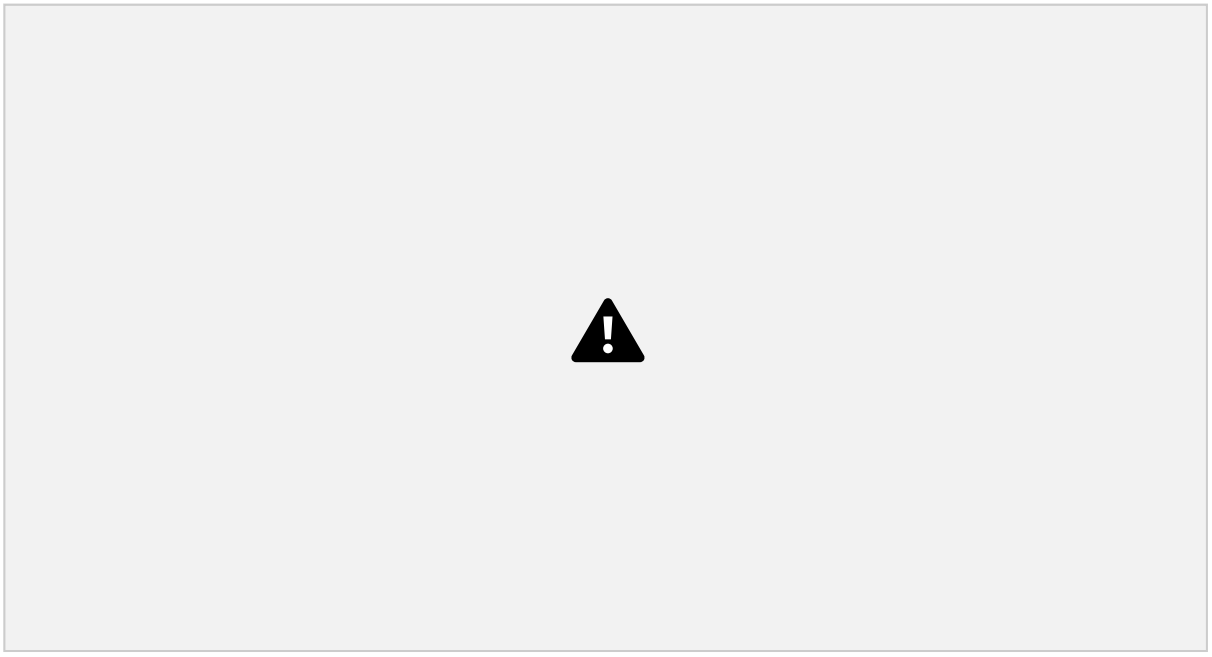
```
#include <reg51.h>

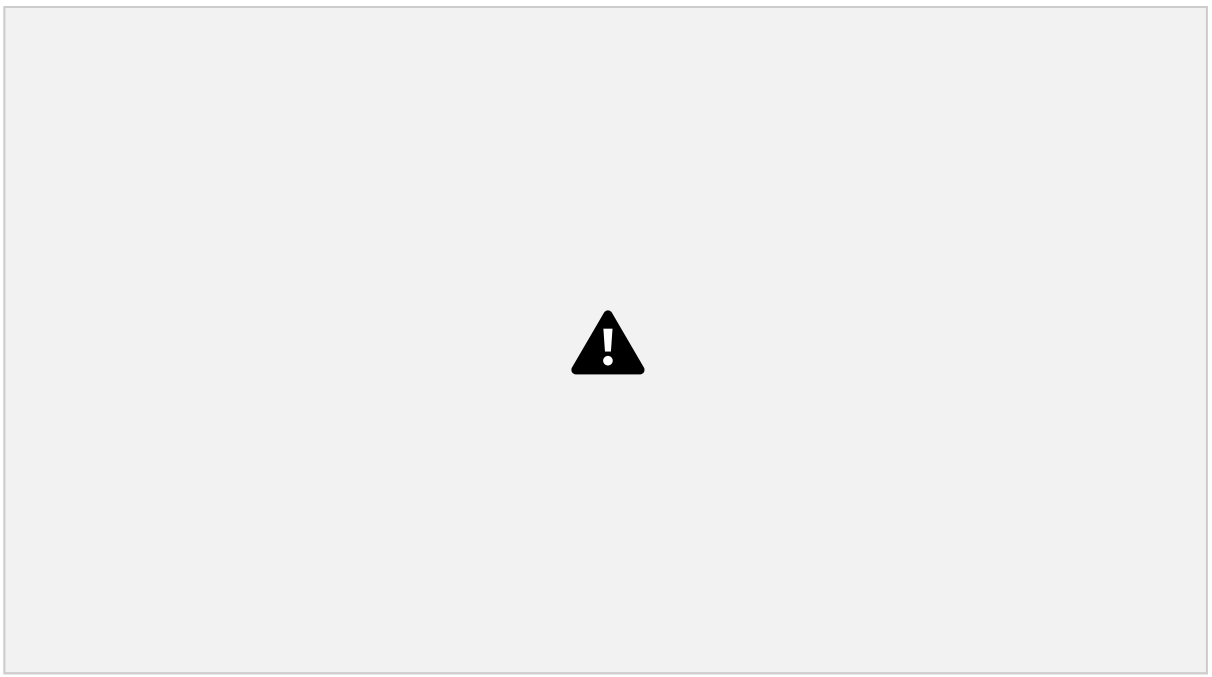
sbit LED1 = P1^0;
sbit LED2 = P1^1;

void delay()
 {
 unsigned int i,j;
 for(i=0;i<200;i++)
 for(j=0;j<1000;j++);
}

void main()
 {
 while(1)
 {
 LED1 = 1;
 LED2 = 0;
 delay();
 LED1 = 0;
 LED2 = 1;
 delay();
 }
}
```

OUTPUT:





OBERVATION:

Registe **Value**P1 P1	0x01 0x02

RESULT:

Thus GPIO and bit addressing was studied and in Port1 the pins P1.0 and P1.1 toggled alternately.

Exp No: 12

EMBEDDED C PROGRAM TO MULTIPLY TWO 16-BIT BINARY NUMBERS AIM:

To write an Embedded C program to multiply two 16-bit numbers using Keil software

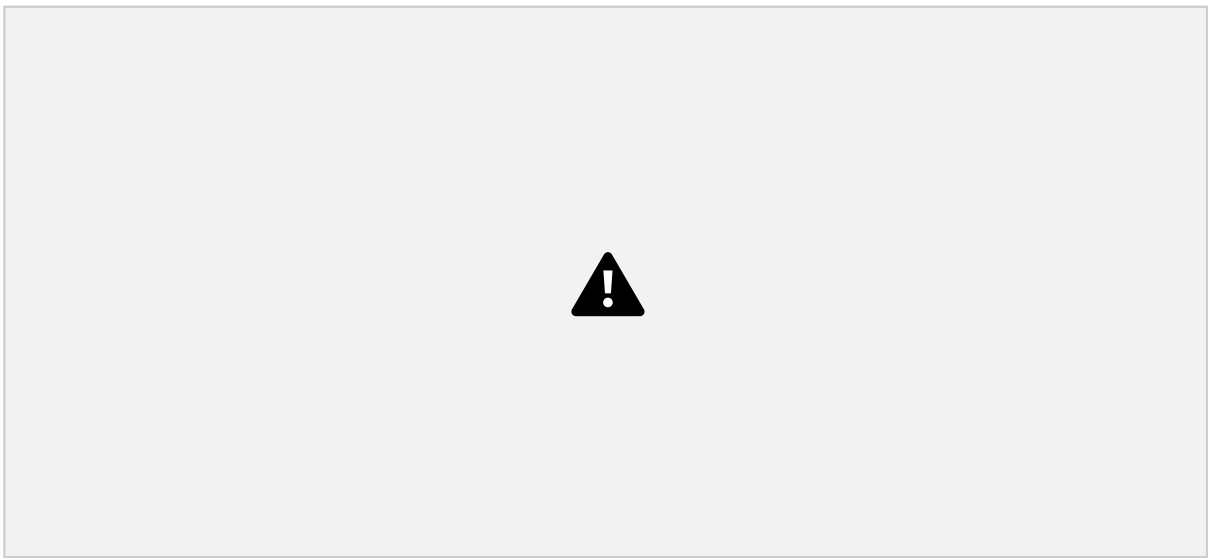
SOFTWARE REQUIRED:

- Keil

Program:

```
#include <reg51.h> void main()
{
while (1)
{
unsigned int num1, num2; unsigned long int product; num1 = 0x2222;
num2 = 0xBBBB;
product = (unsigned long int)num1 * num2; P0= product &0xff;
P1= (product & 0xff00)>>8;
P2 =(product & 0xff0000)>>16; P3 =(product & 0xff000000)>>24;
}
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 13

EMBEDDED C PROGRAM TO ADD AN ARRAY OF 16-BIT NUMBERS AND STORE THE 32-BIT RESULT IN INTERNAL RAM

AIM:

To write an Embedded C program to add an array of 16-bit numbers and store the results in internal RAM using Keil software.

SOFTWARE REQUIRED:

- Keil

PROGRAM:

```
#include <reg51.h>

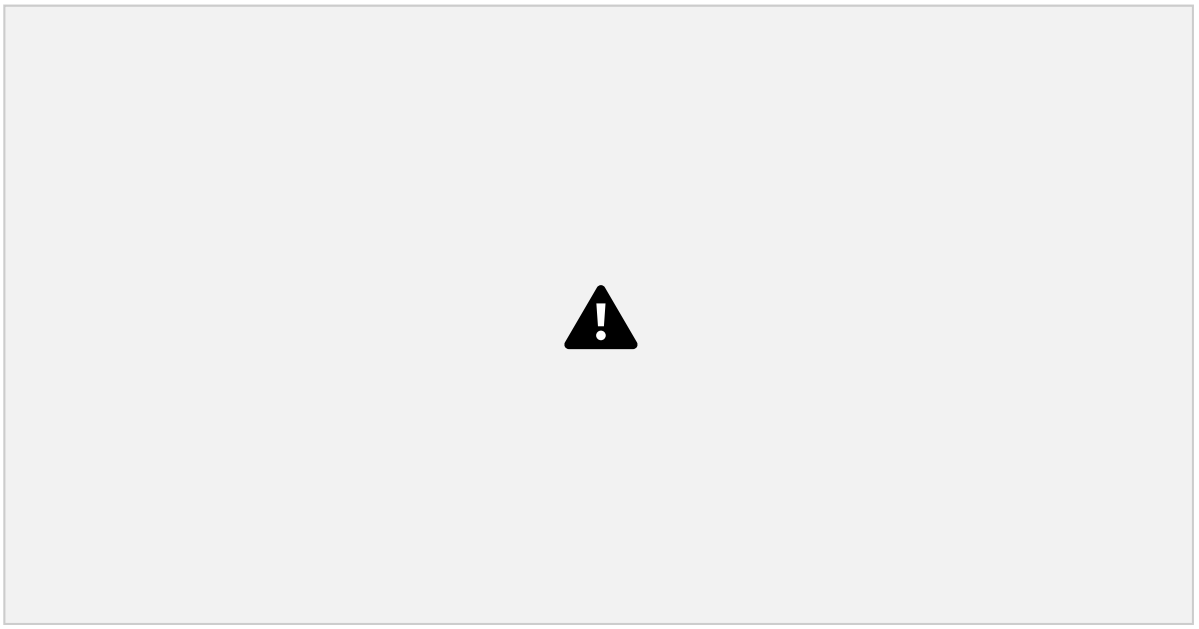
void main(void)
 {
 unsigned int i;
array[00x00 unsigned int array[5] = {0x1111, 0x2222, 0x8888, 0x4444, 0xABCD};
unsigned long sum = 0;

 for (i = 0; i < 5; i++)
 {
 sum = sum + array[i];
 }

 P0 = (unsigned char)(sum & 0xFF); // Lower 8 bits
 P1 = (unsigned char)((sum >> 8) & 0xFF); // Next 8 bits
 P2 = (unsigned char)((sum >> 16) & 0xFF); // Upper 8 bits

 while(1); // keep output stable
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 14

EMBEDDED C PROGRAM TO DISPLAY “HELLO WORLD” MESSAGE

AIM:

To write an Embedded C program to display “HELLO WORLD” message using Keil software.

SOFTWARE REQUIRED:

- Keil sofware

Program:

#include <reg51.h> #include <stdio.h>

void main (void){

SCON = 0x50; /* SCON: mode 1, 8-bit UART, enable rcvr */ TMOD = 0x20; /* TMOD: timer 1, mode 2, 8-bit reload */ TH1 = 0xFD; /* TH1: reload value for 9600 baudrate */

TR1 = 1; /* TR1: timer 1 run */

TI = 1; /* TI: set TI to send first char of UART */ while (1)

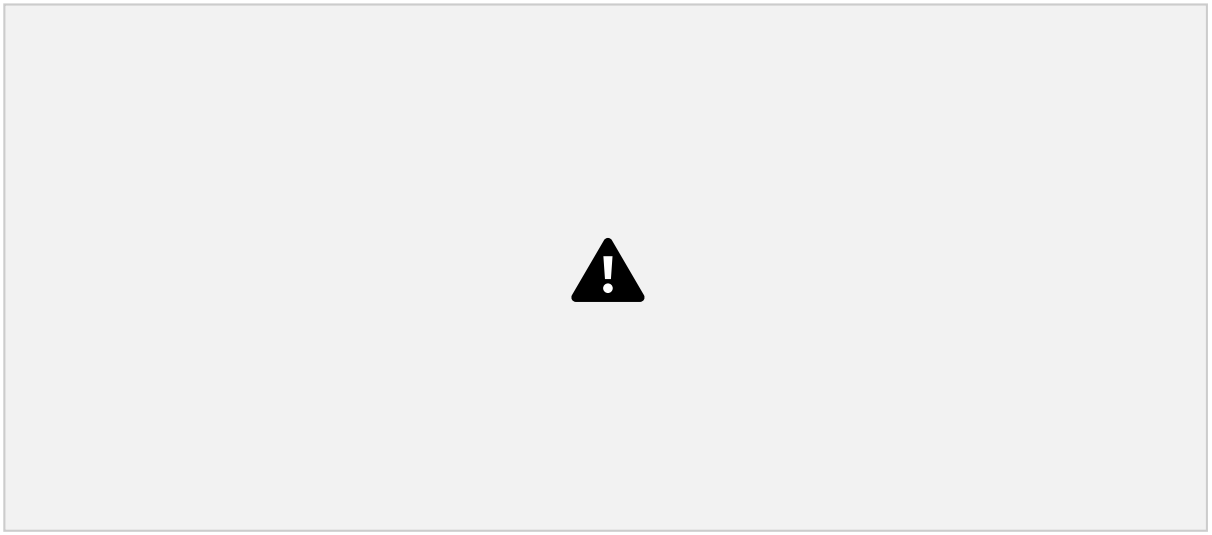
{

printf ("Hello World ! \n ");

}

}

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No: 15

EMBEDDED C PROGRAM TO CONVERT THE HEXADECIMAL DATA 0XCFH TO DECIMAL AND DISPLAY THE DIGITS ON PORTS P0, P1 AND P2 AIM:

To write an Embedded C program to convert Hexadecimal numbers into Decimal and display the digits on ports using Keil software.

SOFTWARE REQUIRED:

- Keil software

PROGRAM:

```
#include <reg51.h>
void main(void)
{
unsigned char hexa = 0x08;
unsigned char hundreds, tens, units;

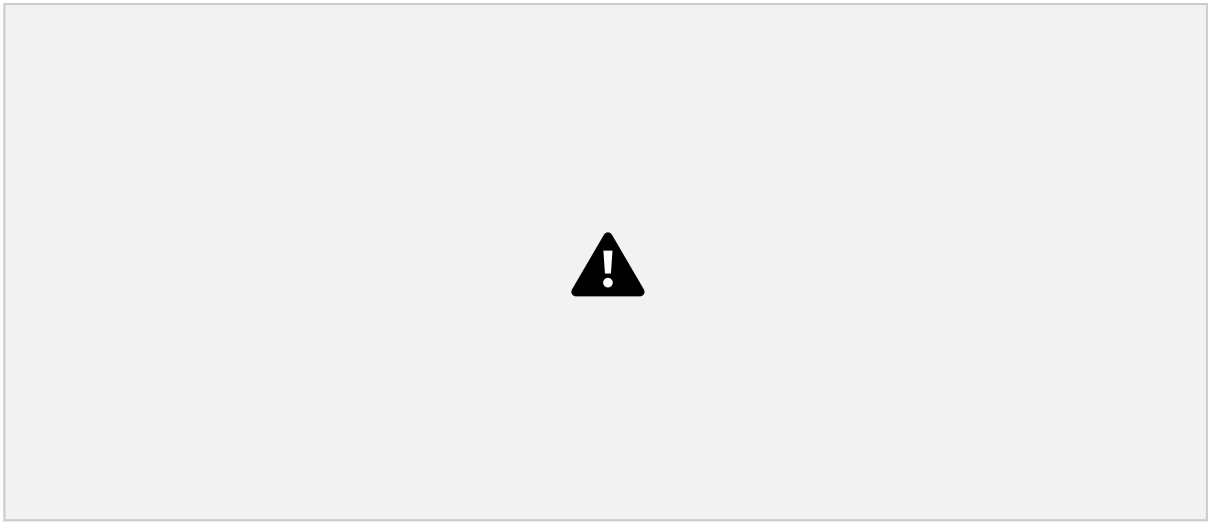
ACC = hexa;
B = 10;
ACC = ACC / B;
units = B;
```

```
B = 10;
ACC = ACC / B;
tens = B;
hundreds = ACC;

P0 = units;
P1 = tens;
P2 = hundreds;

while(1);
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No:16

SERIAL INTERRUPT USING UART (KEYBOARD INPUT) IN

KEIL AIM:

To write and simulate an Embedded C program using **serial interrupt** in 8051 microcontroller such that while a message is continuously transmitted, any key pressed from the keyboard generates an interrupt and is displayed through UART.

APPARATUS / SOFTWARE REQUIRED:

• PC with Windows OS

• Keil µVision IDE

• 8051 Simulator (Keil built-in)

• Serial Window (UART #1)

THEORY:

The 8051 microcontroller provides a **serial interrupt** that occurs when: • **RI (Receive Interrupt flag)** is set after receiving a character, or

• **TI (Transmit Interrupt flag)** is set after transmission.

In this experiment:

• UART is configured in **Mode 1 (8-bit UART)**

• Timer1 is used to generate baud rate (9600)

• Serial interrupt is enabled using ES bit

• When a key is typed in the **Keil Serial Window**, the received character sets RI • The **Serial Interrupt Service Routine (ISR)** is executed immediately

This demonstrates **interrupt-driven serial communication**, which is more efficient than polling.

PROGRAM:

```
#include <reg51.h>
#include <stdio.h>

void serial_ISR(void) interrupt 4
 {
 char ch;
 if (RI)
 {
 RI = 0; // Clear receive interrupt flag  ch =
SBUF; // Read received character

 printf("\n>>> INTERRUPT RECEIVED: ");
printf("%c\n", ch);
```

```
 }
 }

void delay(void)
 {
 unsigned int i;
 for(i = 0; i < 50000; i++); // Delay for visibility }

void main(void)
 {
 SCON = 0x50; // UART mode 1, 8-bit, REN enabled
TMOD = 0x20; // Timer1 mode 2 (auto-reload)  TH1 =
0xFD; // Baud rate 9600
 TR1 = 1; // Start Timer1

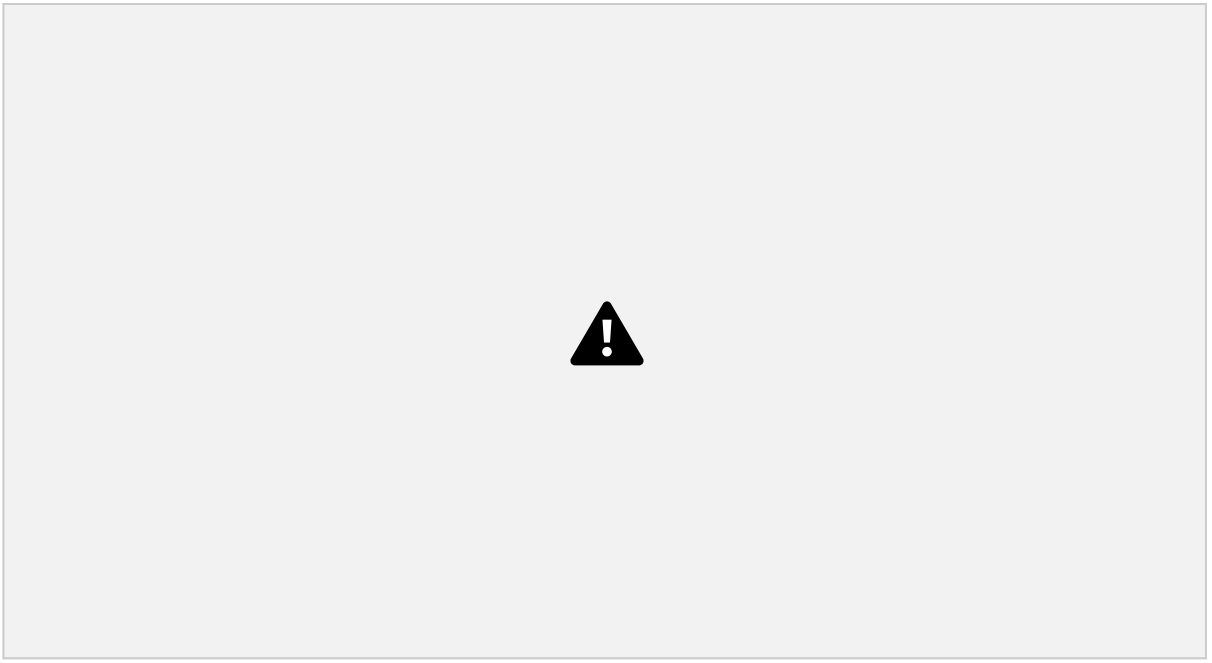
 IE = 0x90; // EA = 1, ES = 1 (enable serial interrupt)  TI

= 1; // Enable printf

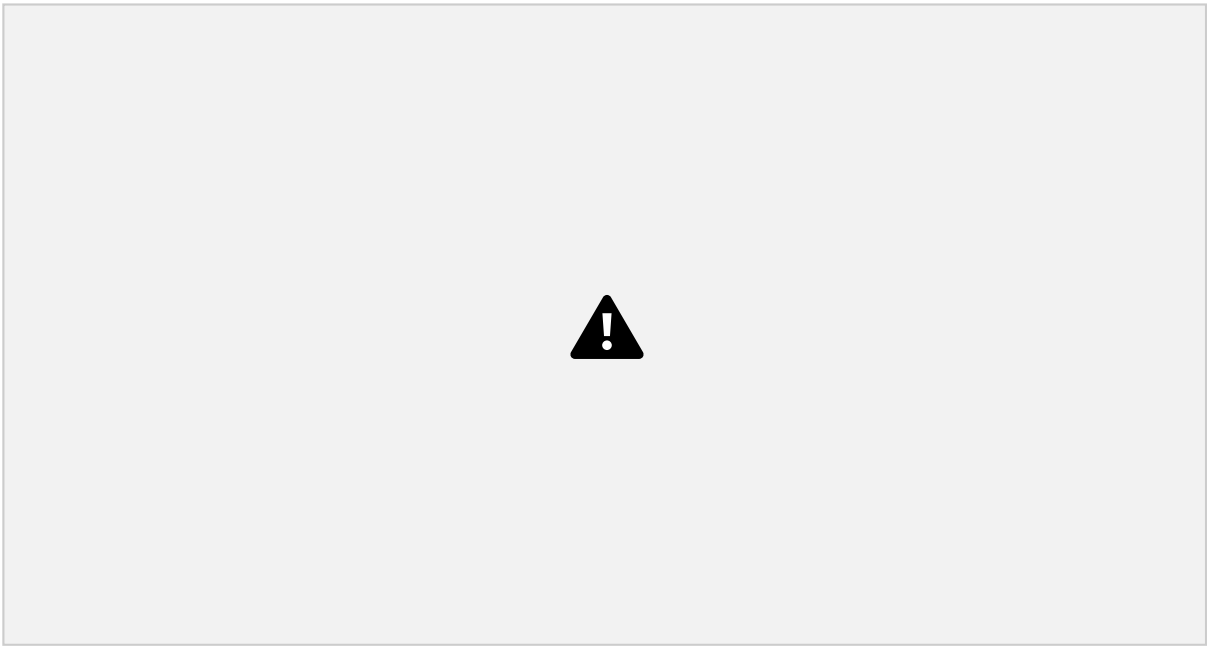
 while(1)
 {
 printf("Hello World\n");
 delay();
 }
 }
```

OUTPUT:

a) Displaying a Message:



b) During interruption:



OBSERVATION:

Action	**ıt in Serial Window** Hello World
Program running	eatedly Key pressed (e.g., A) >>>
	INTERRUPT RECEIVED: A

Action	Output in Serial Window
	pressed Corresponding interrupt

INTERPRETATION:

- Continuous “Hello World” printing indicates **main program execution** •

Typing a key sets the **RI flag**

- Setting RI invokes the **serial interrupt**
- ISR executes immediately and prints the received character
- Interrupt output appears **between normal program outputs**
- This proves that the interrupt **pre-empts normal execution**

RESULT:

Thus, the serial interrupt using UART was successfully implemented and verified in Keil simulator. The interrupt was triggered upon keyboard input and serviced correctly using an Interrupt Service Routine.

Exp No: 17

UART INTERFACING USING INTERRUPT IN

KEIL AIM:

To receive serial data using interrupt.

APPARATUS REQUIRED:

- PC with Windows OS
- Keil µVision IDE
- 8051 Simulator (Keil built-in)
- Serial Window (UART #1)

PROGRAM:

```
#include <reg51.h>

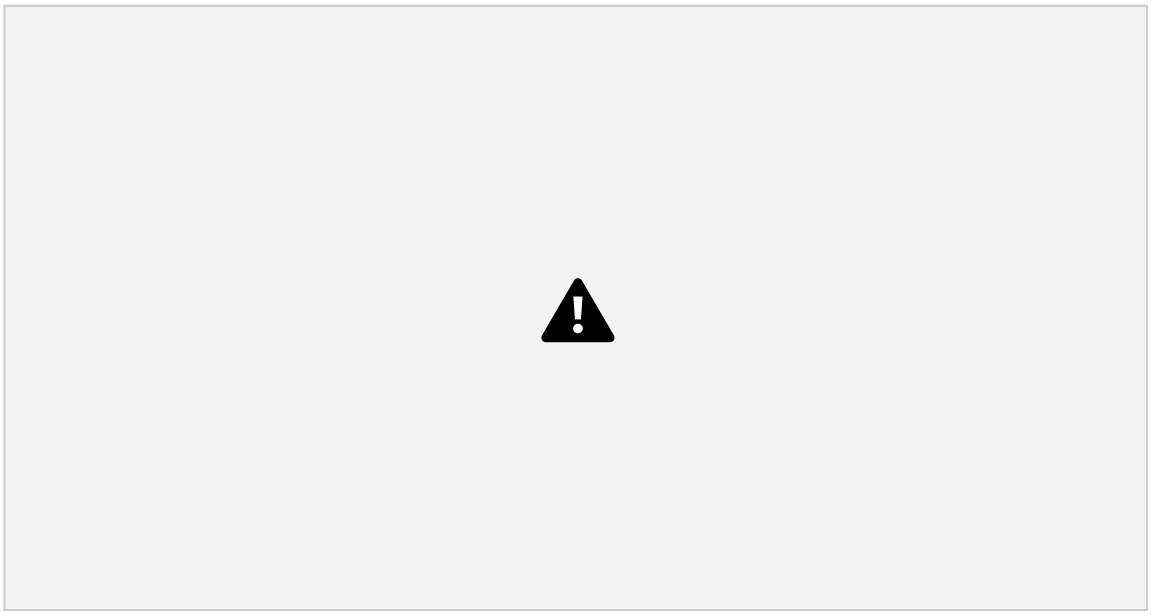
void serial_ISR(void) interrupt 4
 {
```

```
 if(RI)
 {
 P1 = SBUF;
 RI = 0;
 }
 }

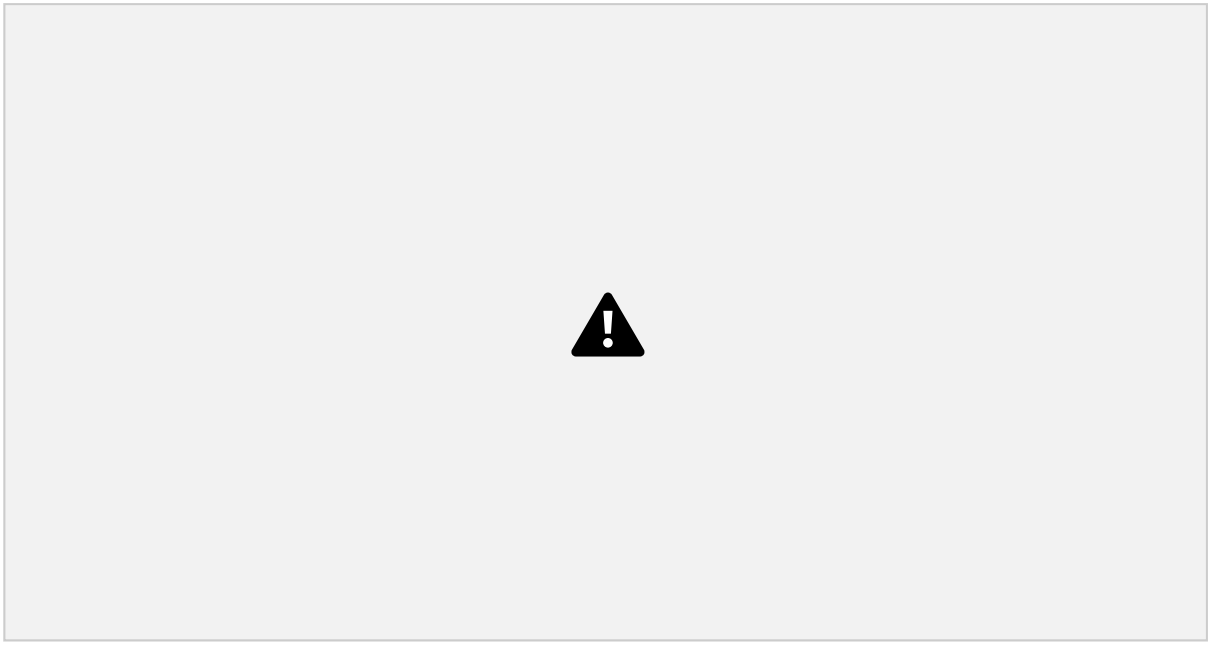
void main()
 {
 TMOD=0x20;
 TH1=0xFD;
 SCON=0x50;
 TR1=1;
 ES=1;
 EA=1;
 while(1);
 }
```

OUTPUT:

Before interrupt:



After interrupt:



OBSERVATIONS:

Input: Character typed in UART window

Processing: Serial interrupt triggered

Output: Port 1 displays ASCII value of received character

A	**I P1**	**0x**
a	**41H**	**41**
1	**61H**	**0x**
	31H	**61**
		0x
		31

Received serial data appears on Port1.

RESULT:

Thus the serial data was received using Interrupt and the received data was verified.

Exp No: 18

GENERATION OF PWM USING TIMER

INTERRUPT AIM:

To generate PWM using Timer interrupt.

APPARATUS / SOFTWARE REQUIRED:

- PC with Windows OS
- Keil µVision IDE
- 8051 Simulator (Keil built-in)
- Serial Window (UART #1)

PROGRAM:

```
#include <reg51.h>

sbit PWM = P1^0;

unsigned char count = 0;
unsigned char duty = 100; // Change this value (0–200)

void timer0_ISR(void) interrupt 1
 {
 TH0 = 0xFC; // Reload for 1ms delay
 TL0 = 0x66;

 count++;

 if(count < duty)
 PWM = 1; // ON time
 else
 PWM = 0; // OFF time

 if(count >= 200) // Total period = 200ms
 count = 0;
 }

void main()
 {
 TMOD = 0x01; // Timer0 Mode 1 (16-bit)
 TH0 = 0xFC; // 1ms preload
 TL0 = 0x66;

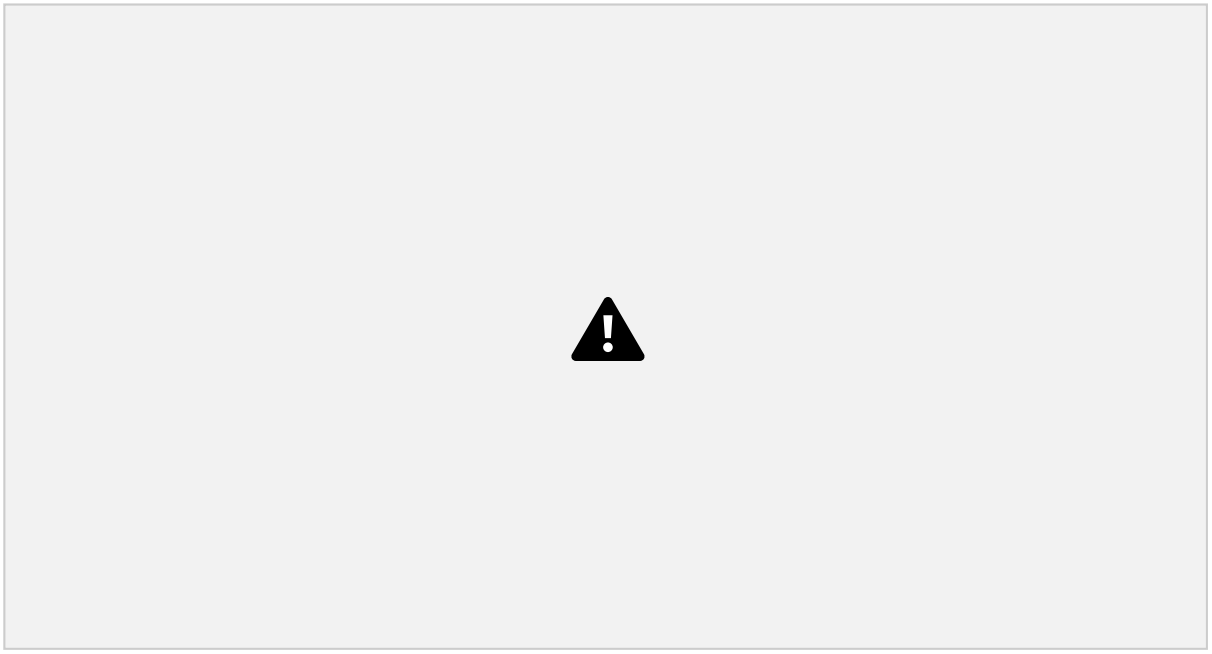
 IE = 0x82; // EA = 1, ET0 = 1
 TR0 = 1; // Start Timer0

 while(1);
```

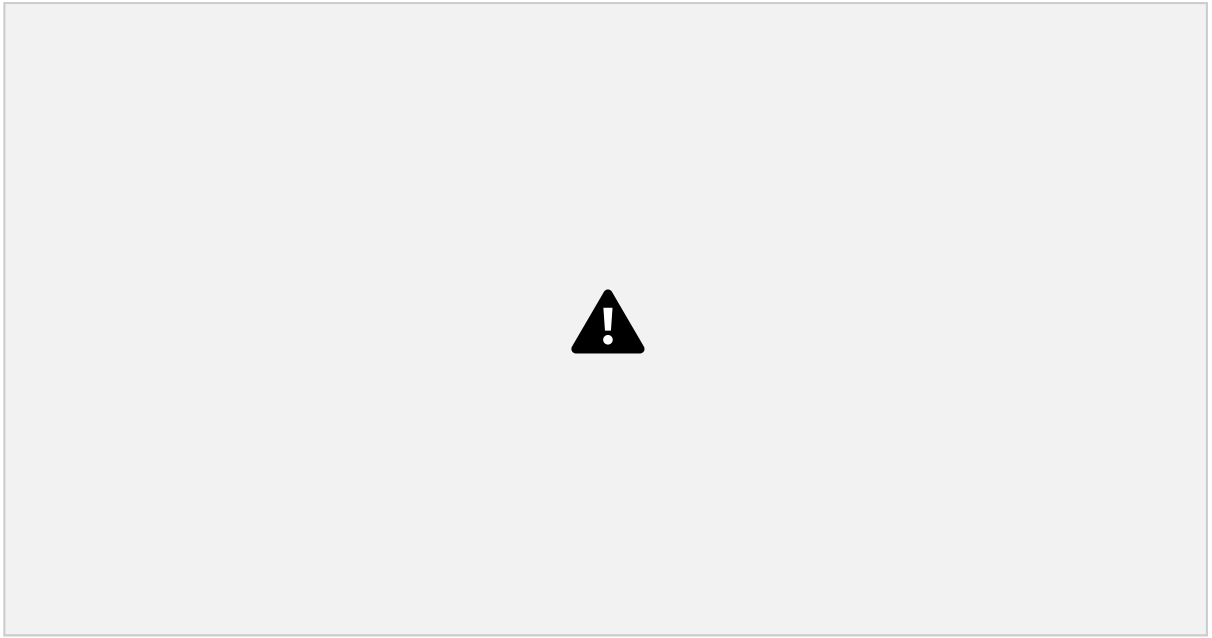
}

OUTPUT:

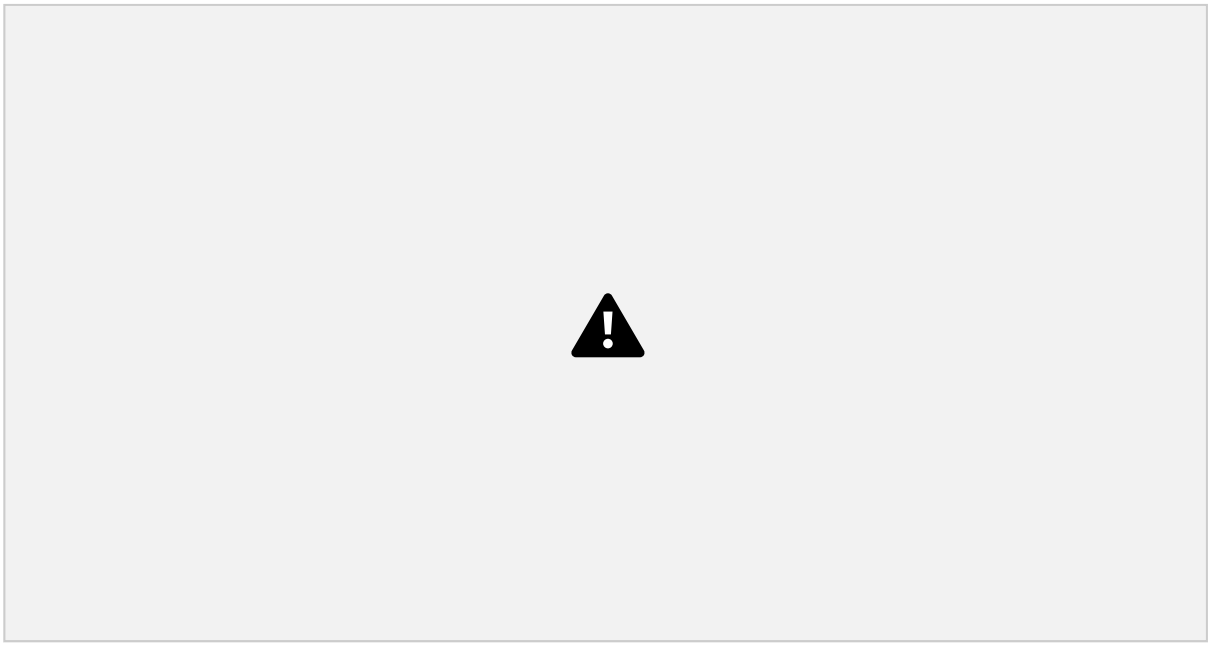
50% Duty cycle:



25% duty cycle:



75% duty cycle:



How to Change Duty Cycle:

Total period = 200 ms
Interrupt = 1 ms

Duty Valu	25%
Cycle50	50%
100	75%
150	100%
200	0%
0	

Formula: RESULT:

Duty cycle (%) = (ON time / Total period) × 100

Thus the PWM was generated using Timer Interrupt for various duty cycle.

STUDY OF ARM PROCESSOR

AIM:

To study the architecture and working of an ARM processor.

COMPONENT REQUIRED:

- ARM LPC2148

THEORY:

ARM is a family of instruction set architectures for computer processors based on a reduced instruction set computing (RISC) architecture developed by British company ARM Holdings. A RISC-based computer design approach means ARM processors require significantly fewer transistors than typical processors in average computers. This approach reduces costs, heat and power use. These are desirable traits for light, portable, battery-powered devices—including smartphones, laptops, tablet and notepad computers), and other embedded systems. A simpler design facilitates more efficient multi-core CPUs and higher core counts at lower cost, providing higher processing power and improved energy efficiency for servers and supercomputers.

Features of LPC214x Series Controllers:

• 8 to 40 kB of on-chip static RAM and 32 to 512 kB of on-chip flash program memory.128 bit wide interface/accelerator enables high speed 60 MHz operation. • In-System/In-Application Programming (ISP/IAP) via on-chip boot-loader software. Single flash sector or full chip erase in 400 ms and programming of 256 bytes in 1ms. • Embedded ICE RT and Embedded Trace interfaces offer real-time debugging with the on-chip Real Monitor software and high speed tracing of instruction execution. • USB 2.0 Full Speed compliant Device Controller with 2 kB of endpoint RAM. In addition, the LPC2146/8 provides 8 kB of on-chip RAM accessible to USB by DMA. • One or two (LPC2141/2 vs. LPC2144/6/8) 10-bit A/D converters provide a total of 6/14analog inputs, with conversion times as low as 2.44 us per channel. • Single 10-bit D/A converter provides variable analog output.

• Two 32-bit timers/external event counters (with four capture and four compare channels each), PWM unit (six outputs) and watchdog.

• Low power real-time clock with independent power and dedicated 32 kHz clock input. • Multiple serial interfaces including two UARTs (16C550), two Fast I2C-bus (400 kbit/s), SPI and SSP with buffering and variable data length capabilities. • Vectored interrupt controller with configurable priorities and vector addresses. • Up to 45 of 5 V tolerant fast general purpose I/O pins in a tiny LQFP64 package. • Up to nine edge or level sensitive external interrupt pins available.

• On-chip integrated oscillator operates with an external crystal in range from 1 MHz to30 MHz and with an external oscillator up to 50 MHz.

• Power saving modes include Idle and Power-down.

• Individual enable/disable of peripheral functions as well as peripheral clock scaling for additional power optimization.

• Processor wake-up from Power-down mode via external interrupt, USB, Brown-Out Detect (BOD) or Real-Time Clock (RTC).

• Single power supply chip with Power-On Reset (POR) and BOD circuits – CPU operating voltage range of 3.0 V to 3.6 V (3.3 V ± 10 %) with 5 V tolerant I/O pads. LPC2148 needs the following hardware to work properly:

• Power Supply
• Crystal Oscillator
• Reset Circuit
• RTC crystal oscillator
• UART

Power Supply

LPC2148 works on 3.3 V power supply. LM 117 can be used for generating 3.3 V supply. However, basic peripherals like LCD, ULN 2003 (Motor Driver IC) etc. works on 5V. So AC mains supply is converted into 5V using below mentioned circuit and after that LM 117 is used to convert 5V into 3.3V.

Reset Circuit

Reset button is essential in a system to avoid programming pitfalls and sometimes to manually bring back the system to the initialization mode. MCP 130T is a special IC used for providing stable RESET signal to LPC 2148.

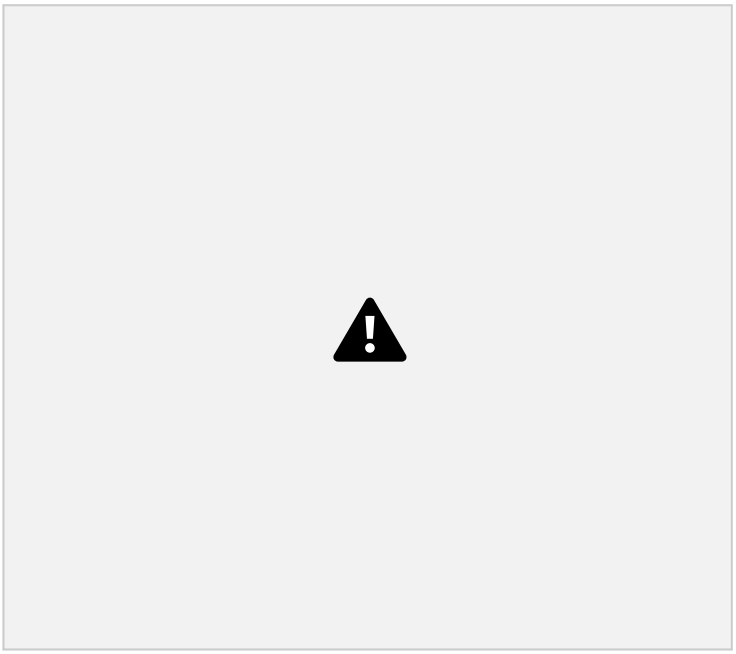


Figure - 3.1: ARM Processor

Flash Programming Utility

NXP Semiconductors produce a range of Microcontrollers that feature both on-chip. Flash memory and the ability to be reprogrammed using In-System Programming technology.

On-board Peripherals

• 8-Nos. of Point LED's (Digital Outputs)

• 8-Nos. of Digital Inputs (Slide Switch)

• 2 Lines X 16 Character LCD Display

• I2C Enabled 4 Digit Seven-Segment Display
• 128x64 Graphical LCD Display

• 4 X 4 Matrix keypad

• Stepper Motor Interface

• 2 Nos. Relay Interface

• Two UART for Serial Port Communication through PC

• Serial EEPROM

• On-chip Real Time Clock with Battery Backup

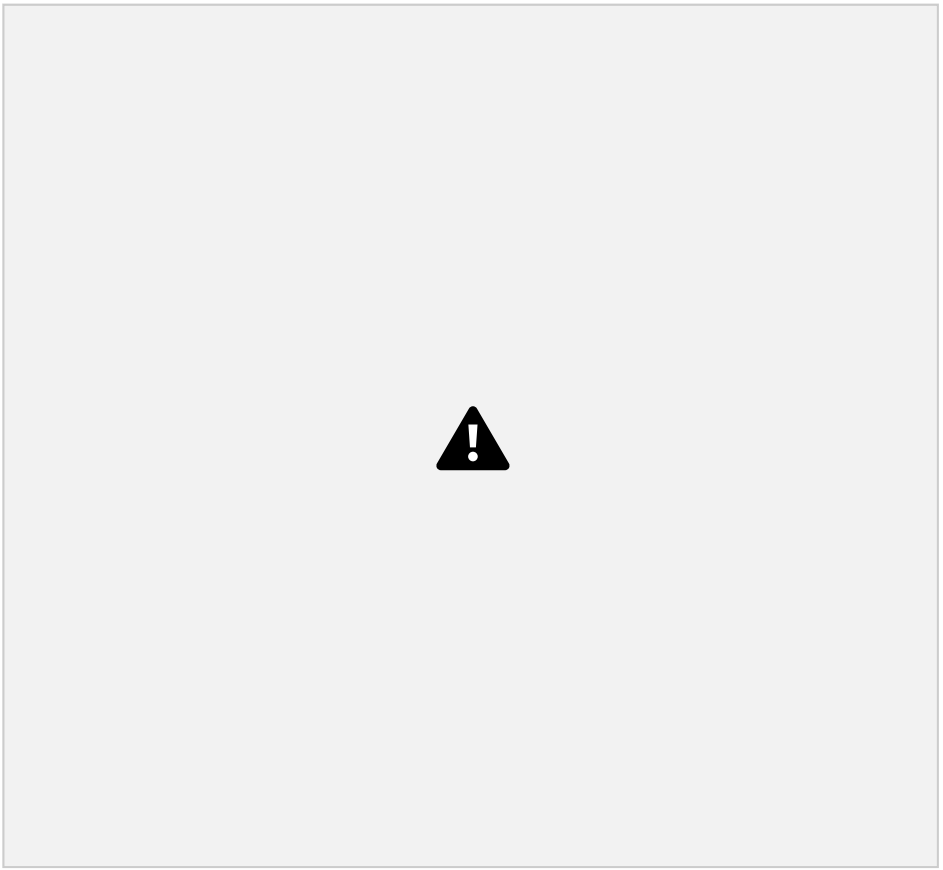
• PS/2 Keyboard Interface (Optional)

• Temperature Sensor

• Buzzer (Alarm Interface)

• Traffic Light Module (Optional)

Pin Configuration



Result:

The ARM processor has been studied successfully.

Exp No: 19

BLINKING LEDS USING SOFTWARE DELAY ROUTINE IN LPC2148 KIT

AIM:

To write and execute a C program to blink LEDs using software delay routine in LPC 2148 kit

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
#include "lpc214x.h"
void delay (unsigned int k);
void main(void)
 {
IODIR0 = 0xFFFFFFFF; //Configure Port0 as output Port
PINSEL0 = 0; //Configure Port0 as General Purpose IO while(1)
 {
IOSET0 = 0x0000ff00; //Set P0.15-P0.8 to '1'
delay(1000); //1 sec Delay
IOCLR0 = 0x0000ff00; //Set P0.15-P0.8 to '0'
delay(1000); //1 Sec Delay
}
}
//Delay Program
//Input - delay value in milli seconds
void delay(unsigned int k)
 {
        unsigned int i,j;
        for (j=0; j<k; j++)
                for(i = 0; i<=800; i++);
}
```

OUTPUT: LEDs P0.15-P0.8 are blinking

RESULT:

Thus the C program to blink LEDs using software delay routine was written and executed in

LPC 2148 kit

Exp No: 20

PROGRAM TO READ THE SWITCH AND DISPLAY IN THE LEDS USING LPC2148 KIT

AIM:

To write and execute C program to read the switch and display in the LEDs using LPC2148 kit

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
#include "lpc214x.h"
int main(void)
 {
 unsigned int sw_sts;
 IODIR0 = 0x0000ff00; //Configure Port0
        PINSEL0 = 0; //Configure Port0 as General Purose IO
while(1)
 {
 sw_sts = IOPIN0;
 IOSET0 = 0x0000ff00; //Set P0.15-P0.8 to '1'
 IOCLR0 = sw_sts >> 8; //Set P0.15-P0.8 to '0'
}
}
```

OUTPUT: LEDs P0.15-P0.08 displayed the bits entered in the switches

RESULT:

Thus C program was written read the switch and display in the LEDs using LPC2148 kit

Exp No: 21

DISPLAY A NUMBER IN SEVEN SEGMENT LED IN LPC2148

KIT AIM:

To write and execute C program to display a number in seven segment LED in LPC2148 kit

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
//SEVEN SEGMENT LED DISPLAY INTERFACE IN C
/* Program to Count 0-9 and Display it in 7 segment Display (MUX) DS4
 * Display Select DS3 ==> "P0.13" Enable --> '0', Disable --> '1'
 * Display Select DS4 ==> "P0.12" Enable --> '0', Disable --> '1'
 */

/* Segment Connection Display 1 & 2 Enable --> '1', Disable --> '0'
 *-----------------------------------------------------------------
 * MSB LSB
 * Dp G F E D C B A
 * P0.23 P0.22 P0.21 P0.20 P0.19 P0.18 P0.17 P0.16
 * 0 0 0 0 0 1 1 0 --> 6 => '1'
 *-----------------------------------------------------------------*/

#include <LPC214X.H>
#define DS3 1<<13 // P0.13
#define DS4 1<<12 // P0.12
#define SEG_CODE 0xFF<<16 // Segment Data from P0.16 to P0.23
unsigned char const seg_dat[]={0x3F, 0x6, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x7, 0x7F, 0x67};
void delayms(int n)
```

```
{
int i,j;
for(i=0;i<n;i++)
 {for(j=0;j<5035;j++) //5035 for 60Mhz ** 1007 for 12Mhz
 {;}
 }
}

int main (void)
{
 unsigned char count;

PINSEL0 = 0; // Configure Port0 as General Purpose IO => P0.0 to P0.15 PINSEL1 =
0; // Configure Port0 as General Purpose IO => P0.16 to P0.31 IODIR0 = SEG_CODE |
DS3 | DS4; //Configure Segment data & Select signal as output IOSET0 = SEG_CODE
| DS3 ; //Disable DS3 display
IOCLR0 = DS4; //Enable DS4 Display
count = 0; //Initialize Count

//Display Count value
IOCLR0 = SEG_CODE;
IOSET0 = seg_dat[count]<<16;

while(1)
 {
 delayms(1000); //1 sec delay
 count++; //Increment count
 if(count>9) count=0; //Limit 0-9
 //Display Count value
 IOCLR0 = SEG_CODE;
```

```
IOSET0 = seg_dat[count]<<16;


}
}
```

OUTPUT: 7-Segment display counting from 0 to 9

RESULT:

Thus C program, was written and executed to display a number in seven segment LED in LPC2148 kit

Exp No: 22

SERIAL TRANSMISSION AND RECEPTION USING ON-CHIP UART IN LPC2148 KIT

AIM:

To write and execute C program for serial transmission and reception using on-chip UART in LPC2148 kit.

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
#include <lpc214x.h>
void UART0_Init(void)
{
 PLL0CON = 0;
 PLL0FEED=0xAA;
 PLL0FEED=0x55;
 VPBDIV = 1;
 // Fpclk = 12.000.000 MHz
 // DLM,DLH = Fpclk / (19200*16) = 39 = 0x27
 PINSEL0 |= 0x5; // Select UART0 RXD/TXD
 U0FCR = 0; // Disable FIFO's
 U0LCR = 0x83; // 8N1, enable Divisor latch bit
```

```
 U0DLL = 0x27; // baud rate fixed to 19200 @ PCLK = 12 Mhz
 U0DLM = 0;
 U0LCR = 3; // Disable Divisor latch bit
}
/*---------------------------------------------------------------*/
/* Function to send one char. to Serial Port */
void sout(unsigned char dat1)
 {
 while(!(U0LSR & 0x20));//Wait for Tx Buffer Empty
 U0THR = dat1; //Send to UART1
}
/*-------------------------------------------------------*/

int main (void)
 { int dat;
 UART0_Init();
 do
 {
 if(U0LSR & 1) /* Check for RDR (Receiver Data Ready)command */
{
 dat = U0RBR;// Receive Data from Srial Port
 sout(dat); // Send Data to Srial Port
          }
 }while(1);
}
```

OUTPUT: Data was serially transmitted

RESULT:

Thus a C program was Written and executed for serial transmission and reception using on chip UART in LPC2148 kit.

Exp No: 23

ACCESS AN INTERNAL ADC AND DISPLAY THE BINARY OUTPUT IN LEDS IN LPC2148 KIT

AIM:

To write and execute C program for accessing an internal ADC and display the binary output in LEDS in LPC2148 kit.

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
#include <LPC214X.H>
#define LEDS 0xFF<<8 //LED => P0.8 to P0.15
////////////////////////////////////////
/*--- ADC Signal Declaration */
////////////////////////////////////////
#define AD0_1 1<< 24
#define CLK_DIV 1<<8
#define PDN 1<<21
#define SOC 1<<24
#define BURST 1<<16
#define DONE 1<<31

/*-------------------------------------------------------------*/
//Delay Program
//Input - delay value in milli seconds
void delay(unsigned int k)
 {
        unsigned int i,j;
        for (j=0; j<k; j++)
```

```
                    for(i = 0; i<=800; i++);
}
/*-------------------------------------------------------------*/
void adc_init()
 {
unsigned long int ADC_CH;

          ADC_CH = 0 | 1 << 1; //Channel AD0.1
 AD0CR = SOC | PDN | CLK_DIV | ADC_CH | BURST ;  }
/*-------------------------------------------------------------*/
unsigned int adc_read( unsigned char channel)  {
          unsigned int aval;
          unsigned long int val;

 if (channel == 1) val = AD0DR1;
 else if (channel == 2) val = AD0DR2;
 else if (channel == 3) val = AD0DR3;

 val = val >> 6;
 val = val & 0x3FF;
 aval = val;
 return (aval);
}
/*-------------------------------------------------------------*/
//////////////////////////
/*----Main Program------*/
//////////////////////////
 int main(void)
 {
unsigned int tp1;
```

```
IODIR0 = LEDS; //Configure Port0 as output Port
PINSEL0 = 0; //Configure Port0 as General Purpose IO
PINSEL1 = 0 | AD0_1; // Enable AD0.1

adc_init(); //Initialise on-chip ADC

do
{ tp1 = adc_read(1); // Channel AD0 0.1
        tp1 = tp1 >> 2; // ADC 10 bit But LED 8bit, Truncate lsb 2 bits
IOSET0 = LEDS; //Switch OFF all LEDS
IOCLR0 = tp1 << 8; //Set VAlue
delay(1000);
}while(1);
}
```

OUTPUT: The Potentiometer knob was adjusted to generate Analog input and Digital display is observed

RESULT:

Thus C program was Written and executed for accessing an internal ADC and display the binary output in LEDS in LPC2148 kit.

Exp No: 24

MODELING CLOCK-DRIVEN CONCURRENT PROCESSES USING SC_THREAD AND WAIT() IN SYSTEMC

AIM:

To understand and implement a clock-driven process using:

- sc_clock
- SC_THREAD
- wait()
- Event sensitivity (positive edge trigger)

in **SystemC**.

APPARATUS/SOFTWARE REQUIRED:

- **Linux OS / WSL & Ubuntu**
- **systemC**

PROGRAM:

```
#include <systemc.h>

SC_MODULE(ClockExample) {
 sc_in<bool> clk;

 void process() {
 while (true) {
 wait();
 std::cout << "Time: " << sc_time_stamp()
 << " - Clock Triggered" << std::endl;
 }
 }

 SC_CTOR(ClockExample) {
 SC_THREAD(process);
 sensitive << clk.pos();
 }
};

int sc_main(int argc, char* argv[]) {

 sc_clock clock("clock", 1, SC_NS);

 ClockExample module("module");
 module.clk(clock);
```

```
 sc_start(5, SC_NS);

 return 0;
}
```

STEPS TO EXECUTE:

Edit file: nano test_systemc.cpp

Build:

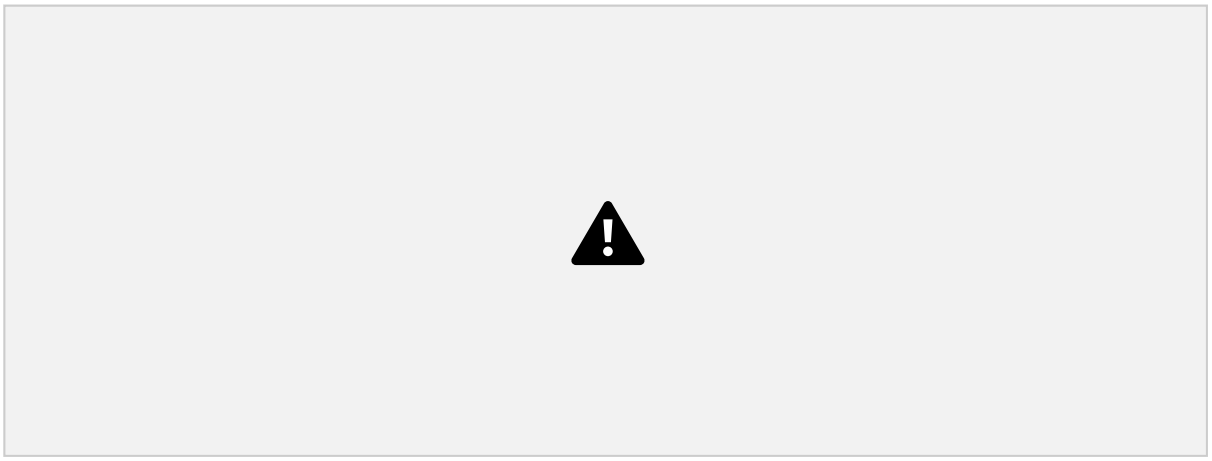
cd build

make

Run:

./test_systemc

OUTPUT:



OBSERVATION:

- Clock period = 1 ns
- Simulation run time = 5 ns
- Thread triggered at every positive edge
- Time increments correctly

RESULT:

Thus, the clock-driven process using SC_THREAD and wait() was successfully implemented and verified in SystemC.

Exp No: 25

PRODUCER–CONSUMER PROBLEM USING SYSTEMC

AIM:

To implement and simulate the Producer–Consumer problem using SystemC.

SOFTWARE REQUIRED:

Ubuntu / WSL Linux

g++ Compiler

CMake
SystemC Library

ALGORITHM:

- Start the simulation.
- Create Producer and Consumer modules.
- Create a FIFO channel of fixed size.
- Connect producer output to FIFO.
- Connect consumer input to FIFO.
- Producer writes data into FIFO.
- Consumer reads data from FIFO.
- Run simulation for fixed duration.
- Stop simulation.

PROGRAM:

```
#include <systemc.h>

// Producer Module
SC_MODULE(Producer) {
 sc_fifo_out<int> out;

 void produce() {
 for (int i = 1; i <= 5; i++) {
 cout << "Produced: " << i << endl;
 out.write(i);
 wait(1, SC_SEC);
```

```
 }
 }

 SC_CTOR(Producer) {
 SC_THREAD(produce);
 }
};

// Consumer Module
SC_MODULE(Consumer) {
 sc_fifo_in<int> in;

 void consume() {
 int value;
 while (true) {
 in.read(value);
 cout << "Consumed: " << value << endl;
wait(1, SC_SEC);
 }
 }

 SC_CTOR(Consumer) {
 SC_THREAD(consume);
 }
};

int sc_main(int argc, char* argv[]) {
 sc_fifo<int> fifo(5);

 Producer producer("Producer");
 Consumer consumer("Consumer");

 producer.out(fifo);
```

```
 consumer.in(fifo);
 sc_start(10, SC_SEC);

 return 0;
}
```

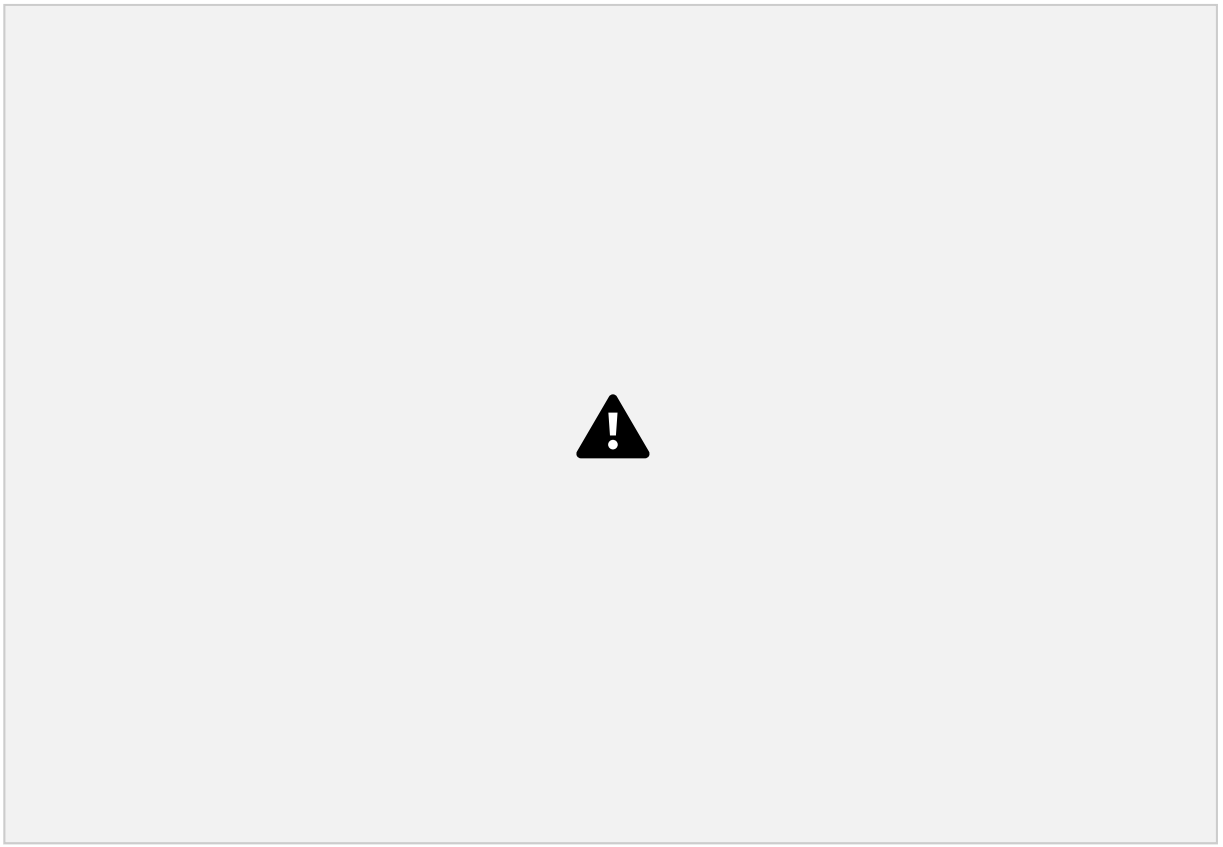
COMPILATION:

```
mkdir build
cd build
cmake ..
make
```

EXECUTION:

```
./producer_consumer
```

OUTPUT:



OBSERVATION:

The producer generates data sequentially and writes it into the FIFO

channel. The consumer reads the data from the FIFO in the same order.

There is no data loss or overlap.

The output shows synchronized execution between producer and consumer.

The wait() function ensures timed execution, and the FIFO automatically handles blocking when necessary.

RESULT:

Thus, the Producer–Consumer problem was successfully implemented and simulated using SystemC. Proper synchronization between concurrent processes was achieved using FIFO communication.

Exp No: 26

TRANSACTION LEVEL MODELING (TLM) USING SYSTEMC

AIM:

To model and simulate a simple Transaction-Level Model (TLM) using Initiator and Target modules in SystemC.

SOFTWARE REQUIRED:

- Ubuntu (or WSL Ubuntu)
- g++
- CMake
- SystemC with TLM-2.0 library

PROGRAM:

```
#include <systemc>
#include <tlm>
#include <tlm_utils/simple_initiator_socket.h>
#include <tlm_utils/simple_target_socket.h>

using namespace sc_core;
using namespace tlm;
using namespace std;
```

```
// Target
struct Target : sc_module {
 tlm_utils::simple_target_socket<Target> socket;

 SC_CTOR(Target) : socket("socket") {
 socket.register_b_transport(this, &Target::b_transport);
 }

 void b_transport(tlm_generic_payload& trans, sc_time& delay) {
 int* data = (int*) trans.get_data_ptr();
 cout << "Target received: " << *data << endl;
 delay += sc_time(10, SC_NS);
 }
};

// Initiator
struct Initiator : sc_module {
 tlm_utils::simple_initiator_socket<Initiator> socket;

 SC_CTOR(Initiator) : socket("socket") {
 SC_THREAD(thread_process);
 }

 void thread_process() {
 tlm_generic_payload trans;
 sc_time delay = SC_ZERO_TIME;

 int data = 555;

 trans.set_data_ptr(reinterpret_cast<unsigned char*>(&data));
trans.set_data_length(sizeof(data));
```

```
 cout << "Initiator sending: " << data << endl;
socket->b_transport(trans, delay);

 wait(delay);
 }
};

int sc_main(int argc, char* argv[]) {

 Initiator initiator("initiator");
 Target target("target");

 initiator.socket.bind(target.socket);
 sc_start();

 return 0;
}
```

COMPILATION STEPS:

Navigate to project directory:

```
cd ~/systemc_test
```

Create build folder (if not already created):

```
mkdir build
cd build
```

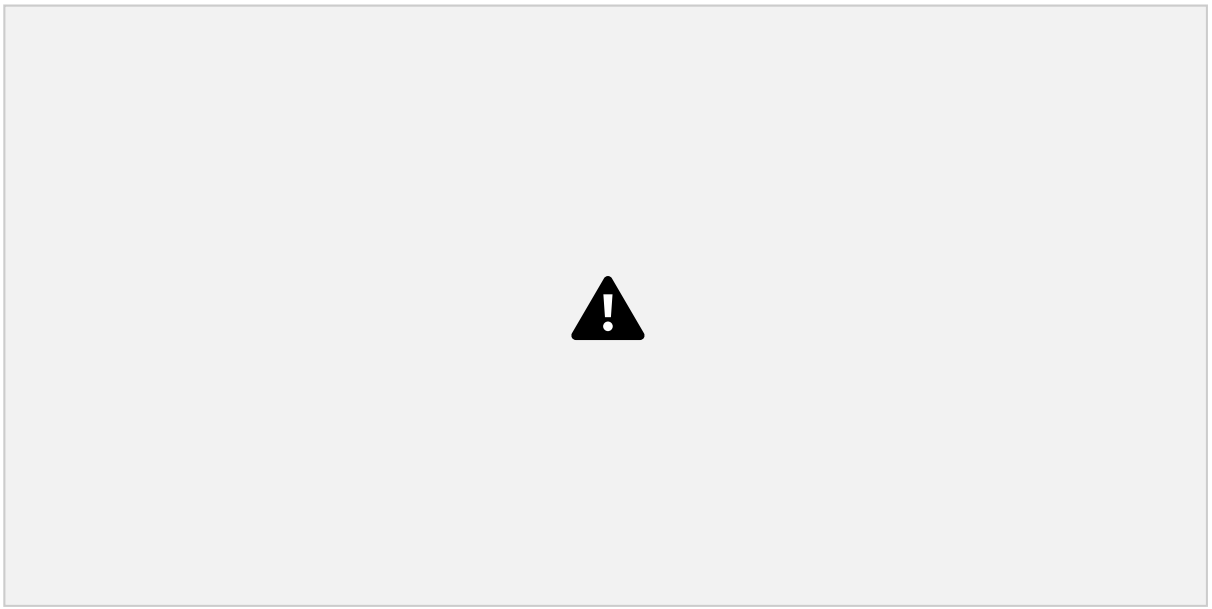
Run:

```
cmake ..
make
```

Execute:

```
./tlm
```

OUTPUT:



OBSERVATION:

The Initiator successfully sends a transaction containing integer data (555) to the Target module. The Target receives the transaction using the b_transport() function and prints the received data.

No clock or signal-level communication is used. Communication occurs at transaction level using sockets and generic payload. The delay of 10 ns is added during transaction execution, demonstrating blocking transport behavior.

RESULT:

Thus the Transaction-Level Model using Initiator and Target modules was successfully implemented and simulated using SystemC.

Exp No: 27

BINARY SEMAPHORE IN SYSTEMC

AIM:

To implement and simulate a Binary Semaphore using SystemC for task synchronization.

SOFTWARE REQUIRED:

- Ubuntu (or WSL Ubuntu)
- g++
- CMake
- SystemC

ALGORITHM:

1. Initialize semaphore = 1
2. Task1 tries to acquire semaphore
3. If available → enter critical section
4. If not → wait
5. Release semaphore
6. Notify waiting process

PROGRAM:

```
#include <systemc.h>

SC_MODULE(binary_semaphore) {

 int sem;
 sc_event ev;

 void task1() {
 while(true) {
 wait(1, SC_SEC);

 if(sem == 1) {
 sem = 0;
 cout << "Task1 Entered Critical Section at " << sc_time_stamp() << endl;  wait(2,
SC_SEC);
 sem = 1;
 ev.notify();
 }
 else {
 wait(ev);
 }
 }
 }
```

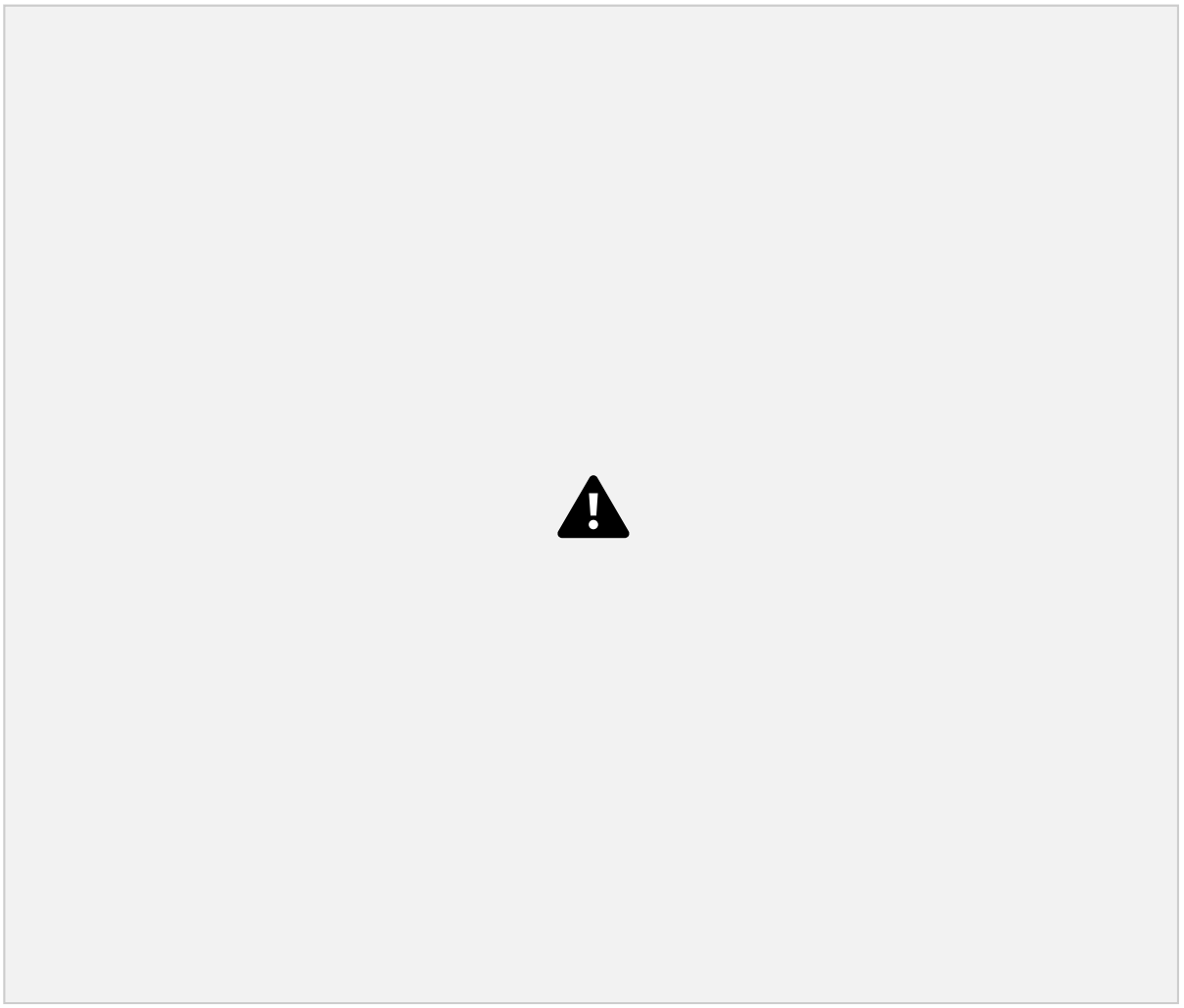
```
 void task2() {
 while(true) {
 wait(2, SC_SEC);

 if(sem == 1) {
 sem = 0;
 cout << "Task2 Entered Critical Section at " << sc_time_stamp() << endl;  wait(1,
SC_SEC);
 sem = 1;
 ev.notify();
 }
 else {
 wait(ev);
 }
 }
 }

 SC_CTOR(binary_semaphore) {
 sem = 1;
 SC_THREAD(task1);
 SC_THREAD(task2);
 }
};

int sc_main(int argc, char* argv[]) {
 binary_semaphore obj("Binary_Semaphore");
 sc_start(10, SC_SEC);
 return 0;
 }
```

OUTPUT:



OBSERVATIONS:

T1			**y Time Exit Time Mutual**
T2	1s	3s	**usion Maintained** Yes
	3s	4s	Yes

✔ Only one task enters critical section at a time.

RESULT:

Thus the Binary Semaphore was created using SystemC and verified.

Exp No:28

COUNTING SEMAPHORE IN SYSTEMC

AIM:

To implement Counting Semaphore allowing multiple resource access.

SOFTWARE REQUIRED:

- Ubuntu (or WSL Ubuntu)
- g++
- CMake
- SystemC

PROGRAM:

```
#include <systemc.h>

SC_MODULE(counting_semaphore) {

 int sem;

 void task() {
 while(true) {
 wait(1, SC_SEC);

 if(sem > 0) {
 sem--;
 cout << "Task Accessing Resource at "
 << sc_time_stamp()
 << " Remaining: " << sem << endl;

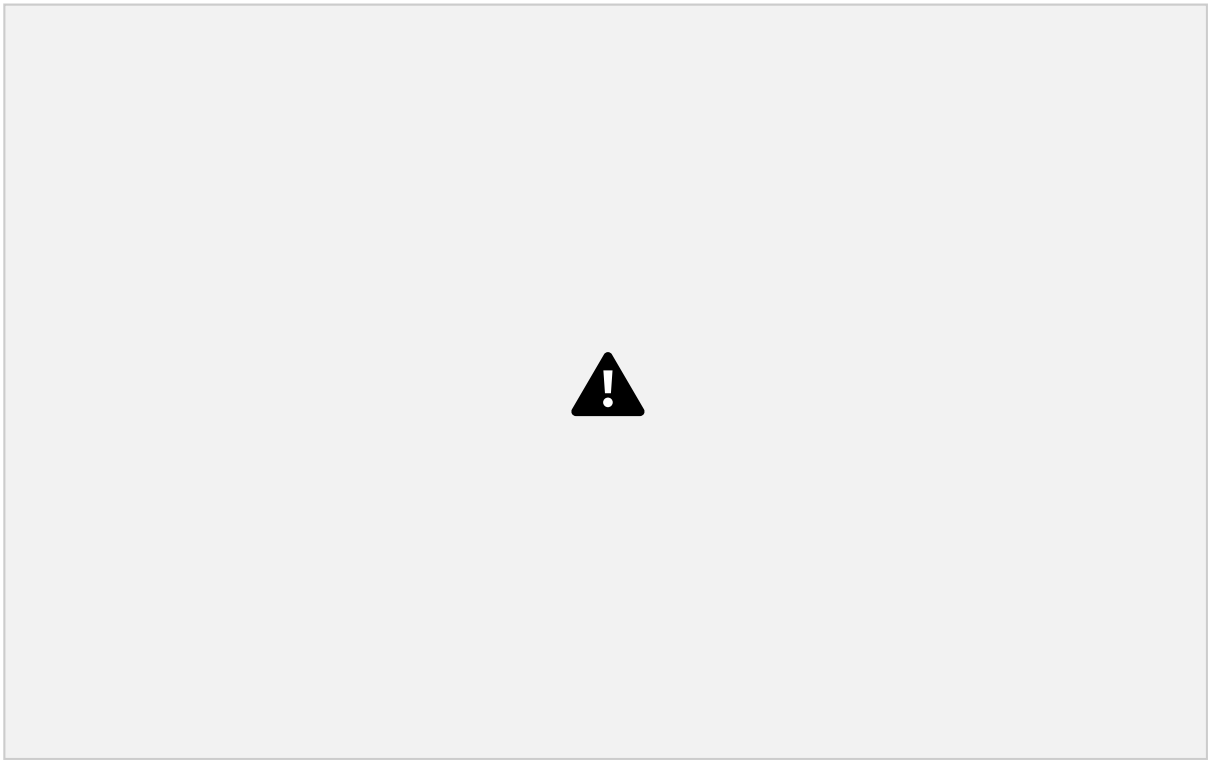
 wait(2, SC_SEC);
 sem++;
 }
 }
 }

 SC_CTOR(counting_semaphore) {
```

```
 sem = 2;
 SC_THREAD(task);
 SC_THREAD(task);
 SC_THREAD(task);
 }
};

int sc_main(int argc, char* argv[]) {
 counting_semaphore obj("Counting");
 sc_start(10, SC_SEC);
 return 0;
}
```

OUTPUT:



OBSERVATION:

- Maximum 2 tasks run simultaneously
- Resource count never negative

RESULT:

Thus the Counting Semaphore was created using SystemC and verified